

Fully Adaptive FROST in the Algebraic Group Model From Falsifiable Assumptions

Ruben Baecker¹, Paul Gerhart², Davide Li Calsi², Luigi Russo², Dominique Schröder², and Arkady Yerukhimovich³

¹ Friedrich-Alexander-Universität Erlangen–Nürnberg, Erlangen, Germany

² TU Wien, Vienna, Austria

³ George Washington University, Washington, D.C., United States

Abstract. We present the first round-optimal Schnorr threshold signature scheme that achieves full adaptive security against algebraic adversaries, relying solely on the Algebraic One-More Discrete Log (AOMDL) assumption. Our scheme, FaFROST, builds on the FROST framework preserving its two-round signing structure and communication efficiency. By avoiding binding commitments to partial public keys, FaFROST circumvents the recent impossibility results from CRYPTO’25 and requires no reliance on the newly introduced, tailor-made LDVR assumption. This establishes that round-optimal, adaptively secure Schnorr threshold signatures are achievable under well-established algebraic assumptions.

1 Introduction

A *threshold signature scheme* allows any subset of t out of n parties to jointly compute a signature on a message m that verifies under a single public key pk [27, 28]. Such schemes are fundamental building blocks for distributed cryptographic protocols, as they enable robustness against up to $t - 1$ corrupted parties while producing signatures that are indistinguishable from those generated by a single signer. They are widely used in blockchain consensus protocols, threshold wallets, and secure key management systems, where a single signing key must remain distributed without sacrificing verifiability or efficiency. Due to their growing importance, NIST is currently preparing a call for the *standardization of threshold signatures*, focusing on schemes that (a) are compatible with the verification algorithm of regular Schnorr signatures [49], and (b) achieve adaptive security. In fact, the call explicitly states [9]:

Given the possibility of adaptive corruptions in the real world, it is important to consider [...] whether the major safety properties of interest (such as unforgeability) are safeguarded against such an adversary.

This motivation has sparked a line of research focusing on Schnorr-based threshold signatures, with *FROST* [43] being at its forefront. FROST, proposed by Komlo and Goldberg, is a Schnorr-based threshold signature scheme that achieves static security and is currently undergoing standardization by the IETF [17]. Due to its clean design and efficiency, several works have investigated whether FROST can also be proven secure against *adaptive* adversaries. Unfortunately, the recent works of Crites, Katz, Komlo, Tessaro, and Zhu [20], Crites and Stewart [22], as well as Crites, Komlo, and Maller [19] provide strong evidence that FROST cannot be proven adaptively secure under standard assumptions.

Strong Assumptions. At CRYPTO’25, Crites et al. [20] showed that proving FROST adaptively secure requires the hardness of the so-called *Low-Dimensional Vector Representation (LDVR)* problem and the interactive one-more discrete logarithm (AOMDL) assumption in the algebraic group model (AGM).

The LDVR problem is a tailored hardness assumption that is parameterized by t_c , t , and n , with $t_c \leq t < n$. The parameter n corresponds to the total number of parties, $t + 1$ to the signing threshold, and t_c to the corruption bound. The authors show that the problem is unconditionally hard for up to $t_c \leq t/2$ corrupted parties, and its hardness beyond that bound remains unknown. Furthermore, they prove that this

Table 1. Comparison of threshold Schnorr signature schemes that achieve full adaptive security. * The work of Gerhart et al. [40] needs a periodic update mechanism to handle adversaries with polynomially many messages. † Bacho et al. [2] proposed the same signature scheme concurrently to our work (c.f. Section 1.4).

	Rounds	Model	Assumption
Bacho et al. [4]	5	ROM	DDH
Katsumata, Reichle, and Takemure [42]	5	ROM	DL
Bacho et al. [3]	3	ROM	DDH
Chen [15]	3	ROM	AOMDL
Gerhart et al. [40]	2*	ROM	DDH
Crites et al. [20]	2	AGM+ROM	AOMDL+LDVR
Our work	2	AGM+ROM	AOMDL
Bacho et al. [2] [†]			

assumption is *tightly linked* to the adaptive security of FROST, in the sense that any proof of adaptive security for the scheme would directly imply the hardness of this assumption. This makes LDVR-based approaches less desirable, as the assumption is non-standard, lacks known reductions to well-established problems, and currently offers no evidence of plausibility beyond certain parameter regimes.

Increasing the Round Complexity. An alternative approach towards realizing an adaptively secure variant of FROST is to increase the round complexity, as shown by Chen [15]. The basic idea of this approach is the addition of a masking round, which achieves statistical indistinguishability and enables a proof strategy similar to that of *SPARKLE* [21]. However, increasing the number of rounds is undesirable in practice, as it leads to higher latency, more complex coordination between signers, and reduced robustness in partially asynchronous or high-latency environments. Moreover, additional rounds undermine one of FROST’s main advantages: its simplicity and efficiency in settings such as blockchain validators and hardware wallets, where minimizing interaction is crucial.

Changing the Functionality. A recent work by Gerhart et al. [40] took a different route by slightly modifying the underlying functionality. This modification incorporates a *periodic update mechanism* that is executed jointly by the parties. Within this model, the authors propose a two-move Schnorr threshold signature scheme that is secure under the Decisional Diffie–Hellman (DDH) assumption.

A key technical ingredient in their construction is a simulation-sound non-interactive zero-knowledge (NIZK) [7] proof that allows each signer to prove to the others that the signing nonce has been deterministically derived from its secret and public inputs. This step is essential to prevent rushing adversaries, while preserving the two-move structure of the protocol. To instantiate this proof under minimal assumptions, they employ an inner-product argument with strong security guarantees [11], in the style of Bulletproofs [10]. Although such arguments are often considered heavy in general-purpose settings, the authors show that in relevant parameter regimes, the resulting proofs are compact and computationally lightweight.

However, while changing the functionality is explicitly listed by NIST as one possible approach to achieve adaptive security, it introduces additional overhead and management complexity due to the need for synchronized updates among all parties.

Discussion. To summarize, existing approaches towards achieving adaptive security for Schnorr-based threshold signatures either rely on strong and non-standard assumptions, increase the number of communication rounds, or modify the underlying functionality itself. Each of these directions departs from the clean, efficient, and practical design principles that made FROST attractive in the first place. Consequently, we are left with the following open problem:

Can one construct a round-optimal Schnorr threshold signature scheme that achieves full adaptive security under well-established assumptions and without changing the functionality?

1.1 FaFROST: Fully Adaptively Secure FROST

To address the open problem posed above, we develop a new threshold Schnorr signature scheme that achieves *full adaptive security* while preserving the round-optimal efficiency and simplicity of FROST. Our construction builds on the structural foundations of FROST2 [18] but introduces new techniques that enable a reduction under established assumptions, without relying on novel and tailored hardness assumptions or additional communication rounds. We begin by revisiting the FROST protocol, which serves as the conceptual basis for our scheme and highlights the core challenges that arise when extending static security proofs to the adaptive setting.

FROST. The signing protocol of FROST2 has two rounds and a local combination algorithm that converts partial signatures in a Schnorr signature with respect to a keypair (pk, sk) . Each signer i holds a secret share sk_i of the group signing key sk . In the first signing round, each signer i samples random nonces $(d_i, e_i) \leftarrow \mathbb{Z}_p^2$ and broadcasts the corresponding commitments $(D_i, E_i) = (g^{d_i}, g^{e_i})$. These values are independent of both the signing key and the message. In the second round, each signer aggregates all commitments via

$$D = \prod_{j \in \mathcal{S}} D_j, \quad E = \prod_{j \in \mathcal{S}} E_j,$$

computes a hash challenge $b = H_{\text{non}}(pk, \mathcal{S}, m, \{D_j, E_j\}_{j \in \mathcal{S}})$, and defines the combined nonce as

$$R = D \cdot E^b.$$

Let $c = H_{\text{sig}}(pk, R, m)$ be the Schnorr challenge associated with the commitment R . Each signer then computes its partial signature

$$s_i = c \cdot A_i sk_i + d_i + b \cdot e_i,$$

where A_i is the Lagrange coefficient for index i . Combination sums the partial signatures into $s = \sum_i s_i$. The final signature is (R, s) , which is valid as it satisfies the standard Schnorr verification equation:

$$g^s = R \cdot pk^c.$$

While the signature scheme is conceptually simple, proving its security in the adaptive setting turns out to be challenging. To motivate our approach towards achieving full adaptive security, we first revisit the techniques used in prior work and explain why they are not sufficient to achieve a round-optimal solution from established assumptions. We start with the security proof against a static adversary.

Static security for FROST. Typically, FROST is proven secure under the *algebraic One-More Discrete Logarithm (AOMDL)* assumption [16]. At a high level, the AOMDL assumption asserts that no efficient adversary can compute the discrete logarithms of $q + 1$ challenge group elements while making at most q queries to a discrete logarithm oracle (each accompanied by a valid *algebraic representation* of the queried group element).

Under the AOMDL assumption, the reduction proceeds as follows. The adversary first commits to the set of parties it intends to corrupt in the security experiment. The reduction then embeds the discrete-logarithm challenge into the public key pk and generates $t - 1$ random signing keys, which are provided to the adversary as the secret keys of the corrupted parties. For the remaining honest parties, the reduction computes consistent partial public keys using polynomial interpolation in the exponent. During the signing protocol, the reduction embeds discrete-log challenges into the elements D_i and E_i of the honest signing parties. It then computes the corresponding partial signatures by querying the discrete-log oracle on $pk_i^c \cdot D_i \cdot E_i^b$, where b and c are derived as in the standard signing algorithm.

The adversary eventually outputs a forgery (s^*, R^*) . Using either the Algebraic Group Model or rewinding-based extraction, the reduction can extract the signing key sk from this forgery. With sk , the reduction can solve all embedded discrete-log challenges. Specifically, if the adversary produces a signature forgery after q signing queries, the reduction solves $q + 1$ discrete-log instances using only q oracle queries, which breaks the AOMDL assumption. While this proof strategy establishes the security of FROST against *static* adversaries, it does not extend to the *adaptive* setting.

Fully Adaptive Security from LDVR. Recall that in the static case, the reduction embeds the discrete-log challenge into the global public key and derives the honest parties’ partial public keys by interpolation in the exponent; this is sound because the reduction never needs to reveal the honest parties’ secret keys. In the fully adaptive setting, however, the adversary may corrupt parties at any time, and the reduction must hand out their secret keys on demand. This is precisely where the static embedding fails: once the discrete-log challenge is tied to the derived partial public keys, producing a corrupted party’s secret key would require computing the discrete logarithm of that party’s individual public key. The reduction can generate at most $t - 1$ public keys with known secret keys, but since it does not know in advance which parties will be corrupted, it cannot ensure that these correspond to the corrupted ones. Querying the discrete-log oracle to recover additional secret keys would also reveal (and thereby solve) the global challenge embedded in the public key, leaving nothing to break once the adversary outputs a forgery.

To overcome this limitation, Crites et al. [20] propose a reduction based on the *LDVR* and *AOMDL* assumptions in the *Algebraic Group Model (AGM)*. Their idea is to embed t challenge values into the n partial public keys as *linear combinations* constructed so that every subset of t public keys is linearly independent. This structure guarantees that, after the adversary corrupts up to $t - 1$ parties, the reduction can compute the corresponding secret keys using $t - 1$ queries to the discrete-log oracle, while at least one independent challenge remains embedded in the public keys.

Eventually, when the adversary outputs a forgery (s^*, R^*) together with an algebraic representation of R^* , the reduction rewrites this representation so that it depends only on the partial public keys. If this expression constitutes a new linearly independent combination of the embedded challenges, the reduction, together with the $t - 1$ independent combinations revealed through corruptions, obtains a system of t linear equations, which it can solve to recover all challenge values and thereby break the *AOMDL* assumption.

A gap remains when R^* lies *within the span* of the partial public keys of corrupted parties, meaning that the forgery does not yield a linearly independent combination. In this case, the reduction cannot extract the signing key sk , and the argument fails. To address this, Crites et al. [20] introduced the *LDVR assumption*, which informally states that producing such a linearly dependent combination is computationally hard. This completes the proof idea: either the forgery yields a linearly independent combination and breaks *AOMDL*, or it yields a linearly dependent one and breaks *LDVR*.

The drawback of this approach is that *LDVR* is largely a proof artefact and remains a relatively new and not yet well-understood assumption. Since we aim to avoid relying on *LDVR*, we next explore an approach that achieves full adaptive security without it, but at the cost of an additional round.

Fully Adaptive Security from Blinding. The main challenge in the fully adaptive setting is that the reduction must provide valid signing keys only after the adversary has already seen the corresponding public keys, such that the public keys already commit to the secret keys. To overcome this challenge, Katsumata, Reichle, and Takemure [42], followed by subsequent work [15], introduced a *blinding* technique in the random oracle model. The high-level idea is to blind each protocol message using the random oracle evaluated on a pairwise key shared between each two signers and a unique session identifier. As long as at least one pairwise key remains unknown, the resulting blinded messages are distributed uniformly at random and appear independent of any individual public key.

The core idea is to blind each protocol message so that, on its own, it appears random and does not commit to any private key. When the messages of all signers are aggregated, the blindings cancel out, revealing the correct combined value. To achieve this, every pair of signers shares an additional pairwise key, which they use to derive a pairwise blinding value by querying the random oracle on a unique session identifier and their shared key.

In the reduction, this mechanism allows the simulator to send uniformly random values instead of honestly computed protocol messages, which would otherwise require knowledge of the honest parties’ secret keys. To ensure that the aggregated messages remain consistent, the reduction employs the *last-consistent-signer* technique: it samples random messages for all but the last honest signer and defines the final honest message as the difference between the intended aggregate value and the sum of the others. Each individual message, therefore, appears random, while their aggregate is consistent with the protocol specification. This approach

never requires the reduction to commit to any partial secret key beyond those already revealed through corruptions.

Upon corruption, the reduction samples a fresh signing key for the newly corrupted party. To preserve consistency with previously sent messages, it then programs the random oracle so that the existing blinded messages equal the honestly computed messages combined with the corresponding blinding values. As long as at least one pairwise key remains hidden and session identifiers are unique, such oracle programming can always be performed without conflict.

While this approach achieves full adaptive security, it crucially relies on the uniqueness of session identifiers. If a session identifier is ever reused, the random-oracle programming would become inconsistent. Ensuring uniqueness thus appears to require either maintaining a secure state across sessions or performing an additional round to agree on a fresh identifier, both of which we want to avoid.

Our Idea. We propose a refined blinding strategy that achieves fully adaptive security for FROST in two rounds without relying on the *LDVR* assumption. In contrast to the approach of Katsumata, Reichle, and Takemure, we apply blinding only to those protocol messages that depend on the partial signing keys. In FROST, this integration is natural: the first round messages, which consist of exchanging (D_i, E_i) , are independent of any secret key material. These messages are also uniformly random values, which we repurpose as a session identifier. We can, therefore, leave the first round unchanged.

The second round messages s_i are key-dependent, and we blind them in the same spirit as Katsumata, Reichle, and Takemure, using the first round messages as a session identifier. As described above, the blinding ensures that these key-dependent messages individually appear uniformly random from the adversary's view. Using our refined blinding strategy, we require no additional rounds, since the session identifier is derived directly from the first-round messages. As we show next, this minor modification enables us to prove full adaptive security in only two rounds, without relying on the LDVR assumption, by employing a new proof strategy. To illustrate how minimal the change is, Figure 1 highlights the only modification to the protocol (beyond key generation).

$\text{SignRound}_2(i, sk_i, state_i, pk, m, S, \{\mathbf{pm}_{1,i}\}_{i \in S})$ $\{(D_j, E_j)\}_{j \in S} \leftarrow \{\mathbf{pm}_{1,i}\}_{i \in S}; (d_i, e_i) \leftarrow st_i; st_i \leftarrow st_i \setminus (d_i, e_i)$ $\text{assert } D_i = g^{d_i} \wedge E_i = g^{e_i}$ $D \leftarrow \prod_{j \in S} D_j; E \leftarrow \prod_{j \in S} E_j$ $b \leftarrow H_{\text{non}}(pk S m \{\mathbf{pm}_{1,i}\}_{i \in S})$ $R \leftarrow D \cdot E^b$ $c \leftarrow H_{\text{sig}}(pk R m)$ $B_i^s \leftarrow \sum_{j \in S \setminus \{i\}} H_s(k_{i,j} \{\mathbf{pm}_{1,i}\}_{i \in S}, m, S, pk) \cdot \Delta_{i,j}$ $A_i \leftarrow \text{Lagrange}(i, S)$ $s_i \leftarrow c \cdot A_i sk_i + b \cdot e_i + d_i$ $s'_i \leftarrow s_i + B_i^s$ $\text{return } s'_i$

Fig. 1. Modified second round of FaFROST. Our modifications are marked in *gray*. Here, for all i, j , the value $\Delta_{i,j}$ equals 1 if $i > j$, and -1 otherwise.

1.2 Proving Security

At a high level, our refined blinding strategy should suffice to prove our scheme fully adaptively secure. However, we cannot apply existing proof techniques directly. We now first explain why the last-consistent-signer technique fails in our setting and then introduce a new challenge-embedding strategy. Using this new strategy, we show how to reduce unforgeability against a fully adaptive adversary to the hardness of the AOMDL assumption.

Last-consistent-signer. In the last-consistent-signer technique, the reduction uses a single discrete-log challenge element and a single oracle query to simulate an entire signing session, regardless of the number of honest signers in that session. As explained above, this suffices when all protocol messages are blinded and the random oracle is programmed carefully. In our setting, however, we must embed the discrete-log challenge into both first-round messages (D_i, E_i) , which commits us to a signer’s randomness. If the signer chosen as the last-consistent one is later corrupted after the first round, the reduction must query the discrete-log oracle twice to solve these challenges. More critically, since only two challenges were embedded, no unresolved challenge remains within that signing query, yet the reduction still requires an additional oracle call to produce partial signatures, as it does not know the corresponding signing keys. This results in the reduction making more discrete-log oracle calls than challenges embedded, preventing it from breaking the AOMDL assumption upon forgery. We therefore need a different strategy for embedding discrete-log challenges.

New challenge embedding strategy. To overcome the issue of too many discrete-log oracle calls for insufficient challenges, we modify our embedding strategy by introducing two discrete-log challenges per honest signer and signing session, rather than embedding challenges for only a single signer throughout the entire session. This ensures that when a signer is corrupted after the first round, we can still open the two corresponding discrete-log challenges and maintain consistency with previously published messages. Crucially, even in this case, at least one honest signer per session remains, since the adversary can corrupt at most $t - 1$ parties while each signing session requires t signers to produce a valid signature. Therefore, when executing the second round, at least two honest challenges remain embedded in a random commitment, allowing us to use the discrete-log oracle to compute a single partial signature. As in the last-consistent-signer technique, we can use the blinding of the second protocol message to simulate uniformly random outputs that combine into the expected values for the adversary.

Implications of first-round commitments. Our new embedding approach resolves the challenge of simulating signing and handling corruptions that occur after the first round. What remains is to address corruptions that happen after the second round. This setting differs from prior techniques, where all protocol messages could be made consistent in the event of corruption. In our case, each signer commits to its randomness in the first round. Once the adversary learns a signer’s secret key and all corresponding blinding keys, the reduction must ensure that the second-round message remains consistent with the first-round commitments in a retroactive manner.

To achieve this, we again rely on the discrete-log oracle. When a corruption of signer i occurs, the reduction samples a fresh random signing key $sk_i \leftarrow \mathbb{Z}_p$ and queries the discrete-log oracle to obtain the logarithm of each D_k and each E_k , which was handed out in behalf of signer i in the k -th first-round oracle query. This yields the randomness for this session, which is $d_k + b \cdot e_k$. Following the same blinding-programming strategy as before, the reduction then makes the already-sent protocol message consistent by solving for the blinding term $B_{i,k}^s$ in

$$\text{pm}_{i,2} = sk_i \cdot A_i \cdot c + d_k + b \cdot e_k + B_{i,k}^s$$

and programming the random oracle accordingly. This strategy is effective as long as at least two honest signers remain in the signing set before corruption occurs. If only one honest signer is left, no further corruption can occur, and two unsolved challenge elements remain. Consequently, in every signing session, at least two honest challenges are preserved, and each session requires exactly one discrete-log oracle query to produce the partial signature. Thus, when corruptions occur after either round, the total number of oracle queries is less than or equal to the number of embedded challenges for each signing round.

Why a rewinding-based proof fails. A subtle issue remains to be addressed in order to complete our proof. Depending on the adversary’s strategy, our reduction must extract the signing key sk *without* querying the discrete-log oracle in order to solve one more challenge than the number of oracle queries. Extracting the signing key is typically achieved through rewinding [16].

In a rewinding-based proof, the reduction runs the adversary twice using the same randomness. At a certain point, it alters the output of the random oracle in the second run, causing the rewinded adversary to deviate from its original behavior. This technique, commonly known as *forking*, enables the reduction to leverage the *special soundness* property of the Schnorr protocol by extracting the signing key once the adversary produces two valid forgeries on the same randomness and message. According to the forking lemma [6], this event occurs with sufficiently high probability, allowing the extractor to rewind the adversary and obtain two transcripts with distinct challenges (but for the same nonce R) necessary for key extraction.

Although the forking technique enables the extraction of the signing key and thus solving the discrete logarithm of pk , it is incompatible with our challenge-embedding approach. The core issue is that forking-based proofs effectively allow the adversary to corrupt at least t signers across the two forks, even though each individual run involves corrupting at most $t - 1$ parties. In contrast, our embedding strategy requires that no more than $t - 1$ signers are corrupted *per signing session*.

To understand why our approach fails under forking, consider a session that exists before the fork where no signer is corrupted. After forking, the adversary corrupts one subset of signers in the first branch and the complementary subset in the second. Consequently, the reduction must resolve the challenges corresponding to each subset separately in the respective forks. Additionally, the reduction already relies on discrete-log oracle calls to simulate signatures.

Since the adversary’s corruptions are split across forks, the total number of corrupted signers across the two executions exceeds the per-session corruption bound. This means that after these corruptions, no embedded challenge remains unextracted for the session in question. As a result, the reduction issues more oracle queries than the number of embedded challenges, preventing the AOMDL assumption from being violated upon forgery and causing the reduction to fail.

In other words, while the corruption threshold $t - 1$ acts as a *local bound* within each individual execution, the forking strategy increases the *global bound* on corrupted signers across forks beyond what the embedding can handle, effectively nullifying the reduction strategy.

Extracting the signing key. We resolve this tension by working in the *Algebraic Group Model (AGM)*. In the AGM, the adversary is assumed to output group elements along with an explicit algebraic representation that describes how they were computed from previously obtained elements. For our security proof, this representation may involve the generator g , the combined public key pk , and all previously broadcast first-round messages $D_{i,k}$ and $E_{i,k}$, and takes the form

$$R^* = g^{\alpha_0} \cdot pk^{\alpha_1} \cdot \prod_{1 \leq k \leq q_s} D_k^{\beta_k} \cdot \prod_{1 \leq k \leq q_s} E_k^{\gamma_k},$$

for known coefficients provided by the adversary to the reduction. And our reduction will use this representation to extract the signing key. The open question is how to do this, given that the representation may include discrete-log challenges that the reduction has not yet solved. In addition, the reduction has no remaining oracle queries at this stage, and we cannot directly follow prior extraction methods [21, 20], since our embedding structure differs from those. To successfully extract sk , we first observe that each term involving $D_{i,k}$ and $E_{i,k}$ can be rewritten using only the public key pk , since the reduction knows (unblinded) partial signatures s_{HS} , which satisfy

$$g^{s_{HS}} = pk_{HS}^c \cdot D_k \cdot E_k^b,$$

for some pk_{HS} dependent from pk . We defer the technical details to Section 4, but this rewriting yields a simplified representation of the form

$$R^* = g^\alpha \cdot pk^\beta,$$

with α and β known to the reduction. Moreover, from the validity of the forgery (R^*, s^*) , the reduction also knows that

$$R^* \cdot pk^{c^*} = g^{s^*}.$$

Combining both equations allows the reduction to solve for the secret key as

$$sk = \frac{s^* - \alpha}{\beta + c^*},$$

which yields a solution, if $\beta + c^*$ does not vanish. As we will demonstrate in our security analysis, this is the case with an overwhelming probability.

Breaking the AOMDL assumption. Once we have extracted sk , it remains to show that our reduction breaks the AOMDL assumption. The first discrete-log challenge corresponds to the combined public key pk . When the adversary outputs a valid forgery, the reduction extracts the signing key sk , thereby solving one challenge. Once the signing key is known to the reduction, it can recover all other embedded discrete-log challenges with the remaining discrete-logarithm oracle queries. Crucially, as discussed above, our reduction is constructed such that, for each signing session, the number of embedded challenges is at least as large as the number of discrete-log oracle queries made throughout the security experiment. Consequently, the total number of discrete-log oracle queries is strictly smaller than the number of solved challenges, and the reduction successfully breaks the AOMDL assumption whenever the adversary outputs a valid forgery.

1.3 Identifiable Abort

We have seen that our modified blinding technique enables proving unforgeability against fully adaptive adversaries without introducing additional rounds. However, a drawback of the blinding techniques proposed by Chen [15] and Katsumata, Reichle, and Takemure [42] is that they only allow one to detect whether *someone* misbehaved during the signing protocol, namely by checking whether the final signature verifies, but not to determine *who* misbehaved. Identifying the misbehaving party is highly relevant in practice [48], and this property is known in the literature as *identifiable aborts* [47]. In the following, we show that our updated blinding technique not only achieves round optimality but also suffices to realize efficient identifiable aborts.

In unblinded threshold Schnorr schemes, misbehavior by an individual signer can be detected by verifying each partial signature (D_i, E_i, s_i) with the equation

$$g^{s_i} \stackrel{?}{=} D_i \cdot E_i^b \cdot pk_i^c,$$

where b is the hash-output as above. We cannot follow this verification approach, since (i) our protocol cannot publish binding partial public keys that commit to individual signing keys, and (ii) each s_i is blinded with the pairwise blinding values. Facing a similar challenge, Pino et al. [47] proposed an approach for a lattice-based threshold signature scheme that achieves identifiable aborts while relying on pairwise blinding values in a static security setting. Their technique, however, is not directly applicable to our setting, as it *does not support adaptive security*. This is the case since using their protocol, one cannot maintain consistency between past protocol messages and newly sampled secret keys after corruption. Yet, we can modify their core idea to obtain identifiable aborts for our blinded round-optimal signature scheme with fully adaptive security. Following prior work [20], we assume a public key infrastructure (PKI) and authenticated point-to-point channels between all participants.

High-level idea. To enable identifiable aborts, each signer must be able to verify that the others' outputs are well-formed. Since the first-round messages are unblinded, it suffices to prove the well-formedness of the second-round messages. In particular, each signer should demonstrate that its second-round value is consistent with the publicly known first-round messages. If this can be done efficiently, the protocol can pinpoint the misbehaving participant whenever the final signature fails to verify. Importantly, this mechanism does not

need to be embedded directly into the signing protocol. Instead, the identifiable-abort phase can be invoked only as an additional round *after* a verification failure. This way, honest executions incur no extra cost, while dishonest behavior can be detected and attributed, disincentivizing misbehavior.

As a first step towards identifying aborts, we consider the unblinded values s_i , whose well-formedness we aim to demonstrate. To prove their well-formedness, we enhance our protocol to include hiding Pedersen commitments that commit to the individual secret keys during setup. This modification does not interfere with our proving strategy: Pedersen commitments are perfectly hiding, and since the reduction knows the trapdoor, it can continue to sample signing keys retroactively. However, having the Pedersen-committed partial signing keys and the unblinded first-round messages, the signers can use Chaum-Pedersen proofs [14] to prove the well-formedness of s_i . Such proofs also do not interfere with our fully-adaptive proving strategy, since Chaum-Pedersen proofs are simulation-sound extractable [11], and thus the reduction can simulate these proofs even when a partial signing key is not fixed.

It remains to incorporate the blinding values in order to prove well-formedness for the blinded shares s'_i . Recall that the shares s'_i have the form

$$s'_i = s_i + \sum_{j \in S \setminus \{i\}} B_{i,j}^s \Delta_{i,j},$$

where $B_{i,j}^s \in \mathbb{Z}_p$ is the pairwise identical blinding for the signers i and j . To incorporate the blinding values into our proof of well-formedness, we let each signer commit to each blinding value $B_{i,j}^s$, again using Pedersen commitments. Then, we can enhance the Chaum-Pedersen proofs for proving just the correctness of s_i to now prove the correctness of s'_i w.r.t. these commitments.

This proof and these commitments guarantee the correct behavior of signer i , if the commitments to the blinding values commit to the correct values. This does not hold in general, since an adversary could just use a malformed $C_{i,j}^s$ during signing, faithfully commit to it, and prove that it's s_i contains $C_{i,j}^s$. This way, the proof would be valid, but the partial blindings would not cancel out during signature aggregation. To avoid this, we observe the following about the blinding commitments, which will allow us to argue for their correctness.

- The correctness of s'_i is preserved as long as the pairwise blinding values of two parties cancel out, even if the individual values were computed incorrectly.
- To commit to the blinding values, we can use unique commitments, which ensure that, based on the same pre-shared key, a given blinding value will result in a unique commitment.
- Using these commitments, if the commitments to the blinding between the parties (i, j) match each other, the corresponding blindings will cancel out.
- If two commitments for the same blinding do not match, at least one of the two parties must be malicious. In that case, we can request that the two involved parties reveal their shared pairwise key to identify which participant used an incorrect blinding.

Requesting that the parties reveal their pairwise keys is acceptable, as if such a mismatch occurs, at least one of the two parties is already malicious, and therefore the shared key is already known to the adversary. Second, the approach of deterministically committing to the pairwise blindings is consistent with our equivocal approach, as we can use random oracle programming for the deterministically derived commitment randomness, as explained next.

Our identification protocol. We now provide a more detailed explanation of our protocol. During setup, each signer receives a pedersen-committed partial public key $pk_i = g^{s_{k_i}} \cdot h^{r_i}$, and for each pairwise shared key $k_{i,j}$ a commitment $K_{i,j} = g^{k_{i,j}} \cdot h^{r_{i,j}}$. As explained above, this does not interfere with our fully adaptive strategy since our reduction will know the discrete logarithm between g and h , and then the public commitments can be equivocated. To initiate the identification protocol, each signer broadcasts its local transcript trans_i , which contains all first-round and second-round messages it has sent or received during the protocol session in question. Each signer i then creates deterministic commitments to its pairwise blinding values $B_{i,j}^s$ as

$$C_{i,j} = g^{B_{i,j}^s} \cdot h^{\text{RO}(k_{i,j}, \text{pm}_{i,2}, \text{pm}_{j,2})},$$

where the randomizer is derived from the pairwise key $k_{i,j}$ and both parties' second-round messages via the random oracle. This construction guarantees that honest parties produce matching commitments for the same blinding value. Once these commitments are computed, each signer proves in zero knowledge that its partial signature

$$s_i = sk_i \cdot A_i \cdot c + d_i + b \cdot e_i + B_i^s$$

is well formed with respect to the partial public key commitment pk_i and first-round values D_i and E_i . The relation proven in the proof is

$$g^{s_i} \cdot D_i^{-1} \cdot E_i^{-b} = g^{sk_i \cdot A_i \cdot c} \cdot \prod_{j \in \mathcal{S} \setminus \{i\}} g^{B_{i,j}^s} \quad \wedge \quad pk_i = g^{sk_i} h^{r_i} \quad \wedge \quad \forall j \in \mathcal{S} \setminus \{i\} : \mathcal{C}_{i,j} = g^{B_{i,j}^s} h^{r_{i,j}}.$$

This zero-knowledge proof integrates seamlessly with our overall proof strategy, as Chaum–Pedersen proofs are simulation-sound extractable [26, 41]. Therefore, honest users can simulate this proof efficiently *without* needing to determine the blinding values at the time of proving. Moreover, these proofs do not prove over correct random oracle evaluations, since only the outputs of the random oracle are relevant, but not that these outputs are faithfully derived.

Verification proceeds as follows:

1. If two honest signers received different first-round messages from the same party, that party is immediately identified as malicious. Since communication takes place over authenticated channels, this check can be performed reliably, and other parties can be convinced.
2. If a proof π_i does not verify, party i is identified as malicious.
3. If no misbehavior is detected in the previous steps, there must exist at least one pair of signers (i, j) whose commitments $\mathcal{C}_{i,j}$ and $\mathcal{C}_{j,i}$ do not match, as otherwise all blindings would cancel and the overall signature would verify. In this case, each pair (i, j) with mismatching commitments reveals their pairwise key, and opens the public commitment $K_{i,j}$ to that key created during setup. Revealing this key is harmless since at least one of the parties in each pair must be malicious anyway.

In this way, misbehavior by any party during the protocol can be efficiently detected in an additional identification round.

1.4 Related Work

Threshold signature schemes have been extensively studied across a wide range of settings, beginning with foundational works [25, 37, 51]. Subsequent research has explored constructions for different signature families, including BLS-based schemes [8], (EC)DSA-based protocols [33, 32, 44, 29, 23, 31, 12, 13, 24, 52, 1], and Schnorr-based variants [34, 36, 35, 43]. Among the latter, statically secure designs include FROST [43, 5, 16] and SPARKLE [21], while recent works have also established strong impossibility results for achieving full adaptive security [19, 20, 22].

As already mentioned in the introduction, several fully adaptive schemes have been proposed recently. For completeness, we briefly cite them here as well. Crites et al. [20] propose an LDVR-based construction, while Chen [15], Glacius [4], and Gargos [3] by Bacho et al. achieve full adaptive security (in either three or five rounds), from DDH. Additionally, Gerhart et al. [40] present a two-round deterministic scheme, also based on DDH, which has been further generalized in a follow-up work [39]. We also compare these works in Table 1.

Concurrent work. Bacho et al. [2] independently propose the same two-round threshold Schnorr signature scheme and prove its security in the Algebraic Group Model (AGM) under the Algebraic One-More Discrete Logarithm (AOMDL) assumption. They further demonstrate that the AGM is indeed necessary by ruling out any black-box reduction from AOMDL in the Random Oracle Model (ROM), which also shows that our proof is optimal in this regard. Our works differ in two main respects. First, their construction does not include a protocol for identifiable aborts, which the authors leave as an open problem. Second, our proof achieves the same tightness with substantially less formalization overhead.

Game $\text{AOMDL}_{(\mathbb{G}, p, g)}^{\mathcal{A}}$	Oracle $\text{OCH}()$	Oracle $\text{OdLog}(X, \alpha, \{\beta_i\}_{i \in [C]})$
1 : $(x_1, \dots, x_C) \leftarrow \mathcal{A}^{\text{OCH}, \text{OdLog}}()$	1 : $X_C \leftarrow \$ \mathbb{Z}_p$	1 : $Q \leftarrow Q + 1$
2 : return $Q < C \wedge \bigwedge_{i \in [C]} x_i == X_i$	2 : $C \leftarrow C + 1$	2 : return $\alpha + \sum_{i \in C} x_i \cdot \beta_i$
	3 : return g^{X_C}	

Fig. 2. Security games for AOMDL.

Bacho et al. [2] prove their construction in three stages: first, they reduce the advantage of breaking the unforgeability of the construction to the advantage of breaking the unforgeability of an idealized signature scheme. Then, they reduce this advantage to either breaking the AOMDL assumption or a new search problem, the *weighted threshold ROS (wtROS)* problem. Finally, they show that this dedicated search problem is hard for any adversary. We, on the other hand, provide a single, direct proof showing that any fully adaptive adversary breaking the unforgeability of our scheme can be used to break the AOMDL assumption. Instead of introducing an additional search problem to bound the probability that the reduction to AOMDL fails, we bound it directly through a combinatorial argument. We believe this simplification is a significant contribution in its own right.

2 Preliminaries

Notation. We denote by $x \leftarrow y$ the assignment of value y to variable x , and we denote by $x \leftarrow \$ X$ the uniform sampling of x from the set X . We utilize the symbol $\mathbb{G} := (\mathbb{G}, p, g)$ to denote a group description, where $\mathbb{G}$ is a cyclic group of order p , where p is a λ -bit prime, and g is a generator of $\mathbb{G}$. Given an element $X \in \mathbb{G}$, we let $\log_g(X)$ denote the discrete logarithm of X with base g , i.e., $\log_g(X)$ is the value $x \in \mathbb{Z}_p$, such that $X = g^x$.

Algebraic One-More Discrete Logarithm. Let $\mathbb{G}$ be a cyclic group of prime order p with generator g . The Algebraic One-More Discrete Logarithm (AOMDL) assumption [46] is the falsifiable version of the One-More Discrete Logarithm assumption (OMDL). In the OMDL game, an adversary $\mathcal{A}$ interacts with two oracles OCH and OdLog. OCH is challenge oracle, and issues a dLog challenge $Y \in \mathbb{G}$ upon receiving a request from $\mathcal{A}$. The OdLog oracle reads an element $Y \in \mathbb{G}$ and returns its discrete logarithm with respect to a generator g . To win the OMDL game, $\mathcal{A}$ must receive C challenges from OCH and compute all of their discrete logarithms in base g by using return at most $C - 1$ calls to OdLog. The associated assumption states that no PPT $\mathcal{A}$ wins the aforementioned game with non-negligible advantage. To make it falsifiable (in the sense of [38]), we can slightly modify OdLog by forcing $\mathcal{A}$ to provide an algebraic representation of the input Y . That is, if $Y_1, \dots, Y_d$ denote the challenges issued by OCH so far, $\mathcal{A}$ must provide $\alpha, \beta_1, \dots, \beta_d \in \mathbb{Z}_p$ such that:

$$Y = g^\alpha \cdot \prod_{i \in [d]} Y_i^{\beta_i}.$$

Figure 2 provides a detailed description of the associated security games. Similar to OMDL, AOMDL assumption states all PPT $\mathcal{A}$ win the security game with at best $\text{negl}(\lambda)$ advantage, which we define as

$$\text{Adv}_{\mathcal{A}, (\mathbb{G}, g, p)}^{\text{AOMDL}}(\lambda) = \Pr[\text{Game}_{\mathcal{A}, (\mathbb{G}, g, p)}^{\text{AOMDL}} \rightarrow 1].$$

Both assumptions imply the hardness of dLog, and AOMDL implies the hardness of OMDL too.

The Algebraic Group Model. The Algebraic Group Model (AGM) [30] is an extension of the well-established Generic Group Model (GGM) [50, 45]. Both are idealized computational models where efficient adversaries performing computations over a group $(\mathbb{G}, \cdot)$ are able to only perform a subset of all the possible computations.

In the GGM, the adversary is given abstract handles to group elements and is oblivious to their representation. Said handles can be used to combine elements through $\cdot$, and perform equality checks. This limited set of operations enables information-theoretic bounds on the success probability of a target computation. The AGM deviates from the GGM by forcing the adversary to output an algebraic representation of every output of theirs. Similar to AOMDL, the algebraic representation of output Z given previously received group elements $g, Y_1, \dots, Y_d \in \mathbb{G}$ is a list of scalars $\alpha, \beta_1, \dots, \beta_d \in \mathbb{Z}_p$ such that:

$$Z = g^\alpha \cdot \prod_{i \in [d]} Y_i^{\beta_i}.$$

An adversary of this type is often called *algebraic*. Unlike the GGM, the AGM does not allow for unconditional lower bounds, but it instead enables reductions between assumptions.

Two-Rounds Threshold Signatures. We define Threshold Signatures, and specialize to the two-rounds case for the purpose of this work. Consider a tuple of PPT algorithms $\text{TS} = (\text{Setup}, \text{KeyGen}, \text{SignRound}_1, \text{SignRound}_2, \text{Combine}, \text{Verify})$ with the following interfaces:

- $\text{Setup}(1^\lambda)$: reads the security parameter in unary encoding, and outputs public parameters par .
- $\text{KeyGen}(n, t)$: reads the number of users n and the threshold $t < n$. It outputs $(pk, \{sk_i\}_{i \in [n]}, \{pk_i\}_{i \in [n]})$, where pk is the public key, and for each $i \in [n]$ sk_i denotes the partial secret key of user i , while pk_i denotes the corresponding partial public key.
- $\text{SignRound}_1(i, sk_i, pk, \{pk_i\}_{i \in [n]}, m, S)$: reads a signer index i , a secret key sk_i , all the public keys, a message to sign m , and a signer set $S \subseteq [n]$. It outputs a state st_i and first-round message $\text{pm}_{1,i}$ for signer i .
- $\text{SignRound}_2(i, st_i, sk_i, pk, \{pk_i\}_{i \in [n]}, m, S, \{\text{pm}_{1,i}\}_{i \in S})$: reads all the inputs that are supplied to SignRound_1 , plus all the first-round messages $\{\text{pm}_{1,i}\}_{i \in S}$ and a state st_i . It outputs a partial signature s_i .
- $\text{Combine}(m, S, \{s_i\}_{i \in S})$: it reads a signer set, a message, and a set of partial signatures. It outputs a combined full signature σ .
- $\text{Verify}(pk, m, \sigma)$: it reads a message m , a full signature σ , and a public key pk . It returns a bit $b \in \{0, 1\}$.

Correctness. Intuitively, a threshold signature scheme is correct if a signature generated by honest users is accepted by the verification procedure. Formally, TS is perfectly correct if for all λ, n, t , all $\text{par} \leftarrow \text{Setup}(1^\lambda)$ and $(pk, \{pk_i, sk_i\}_{i \in [n]}) \leftarrow \text{KeyGen}(\text{par})$, and all signing sets $S \subseteq [n]$ with $|S| \geq t$

$$\Pr[\text{TS-Correct}(pk, \{sk_i\}_{i \in S}, S, m) \rightarrow 1] = 1,$$

with $\text{TS-Correct}(pk, \{sk_i\}_{i \in S}, S, m)$ defined as in Figure 3.

$\text{TS-Correct}(pk, \{sk_i\}_{i \in S}, S, m)$	
1 :	if $S \not\subseteq [n] \vee S \leq t$
2 :	return $\perp$
3 :	for $i \in S$:
4 :	$(\text{pm}_{1,i}, st_i) \leftarrow \text{SignRound}_1(S, m, i)$
5 :	for $i \in S$:
6 :	$s_i \leftarrow \text{SignRound}_2(i, st_i, sk_i, pk, \{pk_i\}_{i \in [n]}, m, S, \{\text{pm}_{1,i}\}_{i \in S})$
7 :	$\sigma \leftarrow \text{Combine}(m, S, \{s_i\}_{i \in S})$
8 :	return $\text{Verify}(pk, m, \sigma) = 1$

Fig. 3. The TS-Correct procedure.

Fully-Adaptive Security. We define fully-adaptive security through the security game of Figure 4, which adapts the standard definition of [20]. We consider a fully adaptive adversary $\mathcal{A}$ with the ability to trigger signing sessions and corrupt users. This is enabled by providing access to three oracles, namely OSignRound_1 and OSignRound_2 for signing and OCorrupt for corruptions. When querying the first two oracles, $\mathcal{A}$ must specify the index i of a honest signer it wishes to interact with, as well as a message to sign, and a signing set. For OSignRound_2 only it must also specify an id eid to identify the signing session to resume. Upon receiving such a request the challenger emulates a call to SignRound_1 and SignRound_2 respectively for the specified parameters, assuming the requested signer behaves honestly. The corruption oracle OCorrupt allows $\mathcal{A}$ to corrupt a user of its choice. Upon corruption of user i , $\mathcal{A}$ learns its partial secret key sk_i as well as its current state st_i . Furthermore, from that point on i is under $\mathcal{A}$'s full control, meaning that $\mathcal{A}$ dictates the messages issued by i . Furthermore, OCorrupt satisfies up to t_c requests. To win the game $\mathcal{A}$ must output a non-trivial forgery (m^*, σ^*) such that $\text{Verify}(pk, m^*, \sigma^*) = 1$. A forgery is considered trivial if m^* is a message for which $\mathcal{A}$ already completed a signing session. This is formalized in the game by storing in a dedicated list Q the messages for which $\mathcal{A}$ finalized a OSignRound_2 request. This criterion corresponds to TS-UF-0 in the hierarchy of unforgeability notions for threshold signatures provided in [5]. With this in mind, we define fully-adaptive security for threshold signatures as follows.

Definition 1 (Full Adaptive Security). For a threshold signature scheme TS and an adversary $\mathcal{A}$ define

$$\text{Adv}_{\mathcal{A}, \text{TS}}^{\text{adp-TS-UF}}(t_c, t, n) = \Pr[\text{Game}_{\mathcal{A}, \text{TS}}^{\text{adp-TS-UF}}(t_c, t, n) \rightarrow 1].$$

TS is (t_c, t, n) -adaptively secure if for all PPT adversaries $\mathcal{A}$, $\text{Adv}_{\mathcal{A}, \text{TS}}^{\text{adp-TS-UF}}(t_c, t, n) = \text{negl}(\lambda)$. TS is fully-adaptively secure if it is (t, t, n) -adaptively secure for all $t, n \in \mathbb{N}$ such that $t < n$.

$\text{adp-TS-UF}_{\mathcal{A}}^{\text{TS}, n, t}(t_c, t, n)$	$\text{OCorrupt}(i)$
1 : $par \leftarrow \text{Setup}(t, n)$	1 : if $ \text{CORR} \geq t_c \vee i \notin [n]$
2 : $(pk, \{pk_i, sk_i\}_{i \in [n]}) \leftarrow \text{KeyGen}(par)$	2 : $\quad \vee i \in \text{CORR} : \text{return } \perp$
3 : $\text{CORR} \leftarrow \emptyset, \text{HON} \leftarrow [n]$	3 : $\text{CORR} \leftarrow \text{CORR} \cup \{i\}$
4 : $Q \leftarrow \emptyset, Q_{st} \leftarrow \emptyset$	4 : $\text{HON} \leftarrow \text{HON} \setminus \{i\}$
5 : $O \leftarrow (\text{OCorrupt}, \text{OSignRound}_1, \text{OSignRound}_2)$	5 : return (sk_i, st_i)
6 : for $i \in [n] : \text{ctr}_i \leftarrow 0$	
7 : $(m^*, \sigma^*) \leftarrow \mathcal{A}^O(par, pk, \{pk_i\}_{i \in [n]})$	$\text{OSignRound}_1(S, m, i)$
8 : return $(m^* \notin Q) \wedge \text{Verify}(pk, m^*, \sigma^*)$	1 : if $i \notin (S \cap \text{HON}) \vee S \not\subseteq [n] :$
	2 : return $\perp$
$\text{OSignRound}_2(\text{eid}, S, m, i, \{\text{pm}_{1,j}\}_{j \in S})$	3 : $\text{ctr}_i \leftarrow \text{ctr}_i + 1$
1 : if $i \notin (S \cap \text{HON}) \vee S \not\subseteq [n] \vee Q_{st}[i, \text{eid}, 1] = \perp :$	4 : $\text{eid} \leftarrow \text{ctr}_i$
2 : return $\perp$	5 : $(\text{pm}_{1,i}, st_i) \leftarrow \text{SignRound}_1(sk_i, pk_i, pk, S, i, m)$
3 : $st_i \leftarrow Q_{st}[i, \text{eid}, 1]$	6 : $Q_{st}[i, \text{eid}, 1] \leftarrow st_i$
4 : $\text{inp} \leftarrow (i, st_i, sk_i, pk_i, pk, S, i, \{\text{pm}_{1,j}\}_{j \in S}, m)$	7 : return $(\text{eid}, \text{pm}_{1,i})$
5 : $s_i \leftarrow \text{SignRound}_2(\text{inp})$	
6 : $Q \leftarrow Q \cup \{m\}$	
7 : return s_i	

Fig. 4. The adaptive unforgeability security game.

3 Our Scheme

In this section, we describe our two-round threshold Schnorr signature scheme FaFROST. For a high-level overview, we refer to Section 1.1.

Setup(n, t, λ)	KeyGen(par)
1 : $(\mathbb{G}, p, g) \leftarrow \text{GGen}(\lambda)$	1 : $sk \leftarrow \mathbb{Z}_p$
2 : if $n > p$: return $\perp$	2 : $pk \leftarrow g^{sk}$
3 : // Select hash functions	3 : $(\alpha_1, \dots, \alpha_{t-1}) \leftarrow \mathbb{Z}_p^{t-1}$
4 : $H_{\text{sig}}, H_{\text{non}}, H_s : \{0, 1\}^* \rightarrow \mathbb{Z}_p$	4 : $f(x) \leftarrow sk + \sum_{i=1}^{t-1} \alpha_i x^i$
5 : $RO \leftarrow (H_{\text{sig}}, H_{\text{non}}, H_s)$	5 : for $i \in [n]$:
6 : $par \leftarrow (n, t, (\mathbb{G}, p, g), RO)$	6 : $sk_i \leftarrow f(i)$
7 : return par	7 : for $i \in [n]$:
	8 : for $j \in [n] \setminus \{i\}$:
	9 : if $i < j$: $k_{i,j} \leftarrow \{0, 1\}^\lambda$
	10 : else $k_{i,j} \leftarrow k_{j,i}$
	11 : return $(pk, \{sk_i, \{k_{i,j}\}_{j \in [n] \setminus \{i\}}\}_{i \in [n]})$

Fig. 5. Setup and key generation for our scheme FaFROST. $\mathbb{G} = \langle g \rangle$ has prime order p and the AOMDL assumption holds in it. For all $id \in \{\text{sig}, \text{non}, \text{s}, \text{d}, \text{e}\}$, H_{id} is modeled as a random oracle.

Setup. To set up our scheme for parameters (t, n) , we select a public cyclic group $\mathbb{G}$ of prime order p and generator g . We select three distinct hash functions (modelled as random oracles) $H_{\text{sig}}, H_{\text{non}}, H_s$ that map arbitrary-length bitstrings to $\mathbb{Z}_p$ scalars.

Key Generation. This process starts by jointly computing a public key pk for Schnorr signature, and by distributing to each signer i the corresponding secret share sk_i . In particular, sk_i is the value $f(i)$, where f is a secret polynomial f of degree $t - 1$, and such that $f(0) = sk$, with $pk = g^{sk}$. Finally, each signer i is given $n - 1$ λ -bitstrings $(k_{i,j})_{j \in [n] \setminus \{i\}}$ sampled uniformly at random with the constraint that $k_{i,j} = k_{j,i}$, for all i, j .⁴

First Round. This first, message-independent round involves each participant generating two scalar nonces $(d_i, e_i) \leftarrow \mathbb{Z}_p^2$ and publish their corresponding commitments $D_i \leftarrow g^{d_i}$ and $E_i \leftarrow g^{e_i}$. The nonces are stored locally and kept for use in the second round.

Second Round. On input the message m to be signed, the signing group S and their corresponding first-round commitments $\{D_j\}_{j \in S}, \{E_j\}_{j \in S}$, along with the local state, each participant aggregates the first-round commitments into $D \leftarrow \prod_j D_j, E \leftarrow \prod_j E_j$. Then, it derives the scalar nonce b as the output of H_{non} evaluated on the transcript, and defines the Schnorr (aggregated) nonce $R \leftarrow D \cdot E^b$.

Each signer i then derives the partial blinding factors $(B_{i,j})_{j \in S \setminus \{i\}}$ as the output of H_s on input the blinding key $k_{i,j}$ and the first protocol message, along with the message and the public key; then, the

⁴ We remark that our scheme does not publish any partial public keys. For formal consistency with our previous definition, we implicitly set $pk_i = \perp$ for all $i \in [n]$.

<u>Lagrange(S, i)</u>	<u>SignRound₂($i, sk_i, st_i, pk, m, S, \{\mathbf{pm}_{1,i}\}_{i \in S}$)</u>
1 : $\Lambda_i \leftarrow \prod_{j \in S \setminus \{i\}} j / (j - i)$	1 : $\{(D_j, E_j)\}_{j \in S} \leftarrow \{\mathbf{pm}_{1,i}\}_{i \in S}; (d_i, e_i) \leftarrow st_i; st_i \leftarrow st_i \setminus (d_i, e_i)$
2 : return Λ_i	2 : assert $D_i = g^{d_i} \wedge E_i = g^{e_i}$
<u>Verify(pk, m, σ)</u>	3 : $D \leftarrow \prod_{j \in S} D_j; E \leftarrow \prod_{j \in S} E_j$
1 : $(R, s) \leftarrow \sigma$	4 : $b \leftarrow \mathbf{H}_{\text{non}}(pk S m \{\mathbf{pm}_{1,i}\}_{i \in S})$
2 : $c \leftarrow \mathbf{H}_{\text{sig}}(pk R m)$	5 : $R \leftarrow D \cdot E^b$
3 : return $(g^s = RX^c)$	6 : $c \leftarrow \mathbf{H}_{\text{sig}}(pk R m)$
<u>SignRound₁(i)</u>	7 : $B_i^s \leftarrow \sum_{j \in S \setminus \{i\}} \mathbf{H}_s(k_{i,j} \{\mathbf{pm}_{1,i}\}_{i \in S}, m, S, pk) \cdot \Delta_{i,j}$
1 : $(d_i, e_i) \xleftarrow{\$} \mathbb{Z}_p^2$	8 : $\Lambda_i \leftarrow \text{Lagrange}(i, S)$
2 : $D_i \leftarrow g^{d_i}; E_i \leftarrow g^{e_i}$	9 : $s_i \leftarrow c \cdot \Lambda_i sk_i + b \cdot e_i + d_i$
3 : $state_i \xleftarrow{\cup} (d_i, e_i)$	10 : $s'_i \leftarrow s_i + B_i^s$
4 : return (D_i, E_i)	11 : return s'_i
	<u>Combine($pk, \{\mathbf{pm}_{1,i}\}_{i \in S}, \{\mathbf{pm}_{2,i}\}_{i \in S}$)</u>
	1 : $\{(D_j, E_j)\}_{j \in S} \leftarrow \{\mathbf{pm}_{1,i}\}_{i \in S}$
	2 : $D \leftarrow \prod_{j \in S} D_j; E \leftarrow \prod_{j \in S} E_j$
	3 : $b \leftarrow \mathbf{H}_{\text{non}}(pk S m \{\mathbf{pm}_{1,i}\}_{i \in S})$
	4 : $R \leftarrow DE^b$
	5 : $\{s_j\}_{j \in S} \leftarrow \{\mathbf{pm}_{2,i}\}_{i \in S}$
	6 : $s \leftarrow \sum_{j \in S} s_j$
	7 : return (R, s)

Fig. 6. Protocol description of our scheme FaFROST. Changes compared to FROST are marked in *gray*. Here, for all i, j , it holds that $\Delta_{i,j}$ equals 1 if $i > j$, -1 otherwise.

combined blinding B_i^s of signer i is computed as $\sum_j B_{i,j} \cdot \Delta_{i,j}$, where $\Delta_{i,j}$ is equal to 1 if $i > j$ and -1 otherwise.

Finally, the signer i computes the partial signature $s_i \leftarrow c \cdot \Lambda_i sk_i + b \cdot e_i + d_i$, where Λ_i is the Lagrange coefficient of index i , and outputs its blinded version $s'_i \leftarrow s_i + B_i^s$.

Signature Aggregation and Verification. Once a signer receives all partial signatures, the final signature consists of a pair $\sigma \leftarrow (R, s)$, with R defined as above, and s being the sum of all partial signatures $s \leftarrow \sum_{i \in S} s'_i$. The verification procedure is identical to a standard Schnorr signature verification, i.e., one computes $c = \mathbf{H}_{\text{sig}}(pk, R, m)$ and checks that $pk^c \cdot R = g^s$.

4 Security Analysis

We now prove the adaptive security of our scheme under the AOMDL assumption in the AGM. Intuitively, we exhibit a reduction from AOMDL to the adaptive unforgeability of our scheme that leverages an algebraic forger $\mathcal{A}$. The following theorem, which also constitutes the main result of this work, formally captures said intuition.

Theorem 1. Let $H_{\text{sig}}, H_{\text{non}},$ and H_s be hash functions modeled as a single random oracle H_{RO} using domain separation techniques. Let $\mathbb{G}$ be a cyclic group of prime order p with generator g and let λ be the security parameter. For all $t_c < t < n \in \mathbb{N}$, any algebraic PPT adversary $\mathcal{A}$ making at most q_s queries to SignRound_1 and at most $q_{\text{RO}} = q_{\text{sig}} + q_{\text{non}} + q_{H_s}$ random oracle queries, there is a PPT adversary $\mathcal{B}$ against AOMDL that makes at most $2 \cdot q_s$ queries to its oracle, such that

$$\text{Adv}_{\mathcal{A}, \text{FaFROST}}^{\text{adp-TS-UF}}(t_c, t, n) \leq \text{Adv}_{\mathcal{B}, (\mathbb{G}, g, p)}^{\text{AOMDL}}(\lambda) + \frac{2 \cdot q_{\text{sig}}}{p} + \frac{q_{\text{non}} \cdot q_{\text{sig}}}{p} + \frac{q_{H_s} \cdot n^2}{2^\lambda} + \frac{q_{\text{sig}} \cdot t}{p(t - t_c)}.$$

Proof. Let $\mathcal{A}$ be an adversary in the adaptive unforgeability experiment making at most q_s queries to the first-round signing oracle SignRound_1 and up to t_c corruption queries. We construct an adversary $\mathcal{B}$ against the algebraic one-more discrete logarithm (AOMDL) assumption that simulates the view of $\mathcal{A}$. The adversary $\mathcal{B}$ receives $2q_s + 1$ challenge elements

$$X_0, \dots, X_{2q_s}$$

from its AOMDL challenger and is allowed at most $2q_s$ queries to the discrete-logarithm oracle. If two of the received challenge elements equal, then $\mathcal{B}$ can break the AOMDL assumption directly using $2q_s$ oracle calls. This happens with probability at most $2q_s/p$, and hence is negligible. Looking ahead, the reduction embeds the first challenge into the public key pk , while at the j -th call to SignRound_1 it embeds the j -th and the $(q_s + j)$ -th challenges, respectively, into the commitments (D_i, E_i) of an honest signer i , namely:

$$\underbrace{X_0}_{\text{in } pk}, \underbrace{X_1, \dots, X_{q_s}}_{\text{in } D_i}, \underbrace{X_{q_s+1}, \dots, X_{2q_s}}_{\text{in } E_i}$$

Its goal in this proof is to solve more discrete logarithms than the number of oracle queries, using $\mathcal{A}$ as a subroutine. In the following, we detail how $\mathcal{B}$ simulates the setup and each oracle of $\text{Game}_{\mathcal{A}, \text{TS}}^{\text{adp-TS-UF}}$ towards $\mathcal{A}$.

Setup and key generation. The security experiment assumes a trusted key generation phase during which no corruptions occur. The reduction $\mathcal{B}$ sets the global public key to the first AOMDL challenge, namely:

$$pk \leftarrow X_0,$$

and samples for every pair of distinct parties $1 \leq i \neq j \leq n$ a symmetric pairwise key $k_{i,j} = k_{j,i} \leftarrow \mathbb{S}\{0, 1\}^\lambda$. To keep track of the experiment state, $\mathcal{B}$ maintains the following lists:

- HON: initialized as $\{1, \dots, n\}$, containing all honest parties.
- CORR: initialized as $\emptyset$; whenever $\mathcal{A}$ corrupts a party i , the reduction moves i from HON to CORR.
- SR₁: stores the inputs and outputs of first-round signing oracle queries.
- SR₂: stores the inputs and outputs of second-round signing oracle queries.
- SRO: records all random-oracle queries, distinguishing between the different oracles used in the protocol.

This completes the setup and key generation phase, after which $\mathcal{B}$ forwards pk to $\mathcal{A}$. Next, the adversary $\mathcal{A}$ interacts with the experiment through two signing oracles, a corruption oracle, and three random oracles. We now describe how each of these oracles is simulated by $\mathcal{B}$.

Simulating the random oracles. Whenever the adversary $\mathcal{A}$ queries one of the random oracles, the reduction $\mathcal{B}$ first checks whether the same input has been queried before by searching for a matching input-output pair in the corresponding list within SRO. If a match is found, $\mathcal{B}$ returns the previously stored output to ensure consistency. If no such pair exists, $\mathcal{B}$ samples a value uniformly at random from the oracle's range and verifies whether this new output is consistent with the simulation strategy. If any inconsistency arises, the reduction aborts. The following cases describe, for each random oracle, what we would consider an inconsistency.

- Signature oracle H_{sig} . When the adversary inputs a pair (pk, R, m) to H_{sig} , it must also provide its algebraic representation of R with respect to all group elements it has already learned, since the adversary is algebraic. Let α_{pk} denote the coefficient associated with pk in this representation, i.e., the adversary explains $R = pk^{\alpha_{pk}} \cdot \prod_i g_i^{\beta_i}$, where the g_i are all previously seen group elements. The reduction aborts if the first input is the challenge public key pk , and if the oracle output $H_{\text{sig}}(pk, R, m)$ equals $-\alpha_{pk}$. Such a collision occurs with probability at most q_{sig}/p . The second error coefficient of this form appears at the end of this proof, when the adversary successfully guesses the output of H_{non} without querying it.
- Nonce oracle H_{non} . When H_{non} is queried for the first time on input $(pk, S, m, \{\text{pm}_{1,i}\}_{i \in S})$, the reduction computes the corresponding group element R according to the protocol specification. It then evaluates $H_{\text{sig}}(pk, R, m)$. If this query to H_{sig} is not fresh, i.e., if $H_{\text{sig}}(pk, R, m)$ was queried before, the reduction aborts. Since there are at most q_{sig} distinct values R and the adversary can query H_{non} at most q_{non} times, this event occurs with probability at most $(q_{\text{non}} \cdot q_{\text{sig}})/p$.
- Blinding-key oracle H_s . The reduction aborts if $\mathcal{A}$ queries a blinding key $k_{i,j}$ to H_s that has not been previously leaked through corruption. Such an event occurs with probability at most $(q_{H_s} \cdot n^2)/2^\lambda$, since there are n^2 possible keys of length 2^λ and the adversary can query H_s at most q_{H_s} times.

Otherwise, $\mathcal{B}$ records the new input-output pair in SR_0 , and returns the sampled value to $\mathcal{A}$.

Simulating SignRound_1 . When the adversary $\mathcal{A}$ queries the first-round signing oracle $\text{OSignRound}_1(S, m, i)$ with an index i in the k -th first-round oracle query, the reduction $\mathcal{B}$ first checks whether i corresponds to an honest signer, i.e., whether $i \in \text{HON}$. If this is the case, $\mathcal{B}$ returns to $\mathcal{A}$ the k -th pair of unused discrete-logarithm challenges

$$(D_i, E_i) \leftarrow (X_k, X_{q_s+k})$$

Finally, $\mathcal{B}$ records the tuple (k, i) in SR_1 for future reference. We observe that this compact representation allows the reduction to easily retrieve the nonces $D_i \leftarrow X_k$ and $E_i \leftarrow X_{q_s+k}$ output at the k -th call to the oracle.

Simulating SignRound_2 . When the adversary $\mathcal{A}$ queries the second-round signing oracle $\text{SignRound}_2(\dots)$, it provides an index i indicating an honest signer, a message m , a set of participating signers S , and a set of first-round messages $\{\text{pm}_{1,i}\}_{i \in S}$. We refer to the tuple $(\{\text{pm}_{1,i}\}_{i \in S}, m, S, pk)$ as session ID sid for better readability in this proof.

The reduction $\mathcal{B}$ first verifies that i is an honest signer and that there exists a first round execution of the protocol. More formally, $\mathcal{B}$ verifies that $i \in \text{HON}$ and that for some k , the tuple (k, i) exists in SR_1 and $D_i = X_k$ and $E_i = X_{q_s+k}$. Then, $\mathcal{B}$ marks the tuple (k, i) as *used* in the set SR_1 by setting it to (k, i, sid) . This marking is appropriate since an honest signer would never reuse a previously used pair.

If no matching entry is found, $\mathcal{B}$ aborts the oracle query. This behavior is consistent with the real protocol, which rejects inputs unless the first-round messages satisfy $D_i = g^{d_i}$ and $E_i = g^{e_i}$, ensuring that only well-formed and consistent states are processed. Revoking used states is consistent with the protocol description, as each internal state is consumed exactly once. Reusing a state could expose the corresponding signing key, violating both correctness and security of the protocol.

After verifying the input consistency, the reduction $\mathcal{B}$ determines how many other honest signers in the current signing set S are expected to respond after this oracle call. For every honest signer $i \in S \cap \text{HON}$, except for the last honest signer in this session, the reduction samples a random value $s_i \leftarrow \mathbb{Z}_p$ and adds the appropriate blinding contributions from all corrupted signers:

$$s'_i \leftarrow s_i + \sum_{j \in S \cap \text{CORR}} H_s(k_{i,j}, sid) \cdot \Delta_{i,j}.$$

It then returns s'_i to the adversary $\mathcal{A}$ as the partial signature.

For the last honest signer (where *last* refers to the order in which $\mathcal{A}$ makes its oracle calls for the current session sid), the reduction $\mathcal{B}$ ensures that the resulting protocol messages remain consistent. Note that if $\mathcal{A}$ previously sent inconsistent inputs to any honest signer, the consistency step is never triggered: a last honest

signer for a given session sid only exists if all honest parties in S received the same first-round message set associated with sid . If inconsistent first-round messages were sent earlier, then at least one honest signer in S will never respond to the same sid , since its corresponding first-round message was already used in a different session. Therefore, this strategy prevents $\mathcal{A}$ from forcing $\mathcal{B}$ to perform additional discrete-logarithm oracle queries by submitting inconsistent inputs.

If all honest signers in a given session receive the same first-round message, the last honest signer needs to maintain consistency. To achieve consistency, the reduction $\mathcal{B}$ computes the aggregate random commitment associated with all the honest signers in the current signing set S :

$$R_{HS} \leftarrow \prod_{j \in S \cap \text{HON}} D_j \cdot E_j^b, \quad (1)$$

where b is obtained from the random oracle H_{non} as specified in the protocol. It then simulates the computation of the challenge $c = \text{H}_{\text{sig}}(pk, R, m)$ by programming the random oracle at that input point, queries the discrete-logarithm oracle on the value $pk^c \cdot R_{HS}$, and obtains the response $\tilde{s}$. To make it consistent with the effective partial signature s_{HS} on behalf of all the honest signers in S , it then computes

$$s_{HS} \leftarrow \tilde{s} - c \cdot \left(\sum_{j \in S \cap \text{CORR}} \Lambda_j \cdot sk_j \right).$$

Next, $\mathcal{B}$ ensures that the value s_{HS} is consistently shared among all honest signers by setting

$$s_i \leftarrow s_{HS} - \sum_{j \in S \cap \text{HON} \setminus \{i\}} s_j.$$

This guarantees that the sum of all (unblinded) honest pre-signature shares equals the effective partial signature s_{HS} , as required by the protocol. Finally, the reduction $\mathcal{B}$ incorporates the blinding terms. The blinding terms between honest signers are handled implicitly: since the values s_i were chosen uniformly at random but in a way that preserves overall consistency, these already incorporate the effect of pairwise blindings among honest parties. Consequently, the reduction $\mathcal{B}$ does not add explicit blinding terms for honest pairs. However, blindings involving corrupted signers do not cancel, and must therefore be added explicitly to maintain consistency with the adversary's view (as also done above). For each honest signer i , the reduction $\mathcal{B}$ sets

$$s'_i \leftarrow s_i + \sum_{j \in \text{CORR} \cap S, j \neq i} \text{H}_s(k_{i,j}, sid) \cdot \Delta_{i,j}.$$

Our signing strategy yields a perfect simulation of the signing oracles: If at least two honest signers remain, there exists at least one pairwise key $k_{i,j}$ unknown to the adversary. If it would query the oracle H_s on this key, the reduction would abort. Moreover, the session ID $sid = (\{\text{pm}_{1,i}\}_{i \in S}, m, S, pk)$ includes at least one honestly generated random first-round message. Consequently, the random-oracle input $(k_{i,j}, sid)$ was never queried before, and the corresponding output of H_s appears to be uniformly random.

From the adversary's perspective, each honest signer's partial signature is therefore distributed uniformly at random and independent of any secret key. When combined with the corrupted parties' contributions, the aggregate of all partial signatures equals $sk \cdot c + d + b \cdot e$, producing a valid Schnorr signature as required by the protocol. If only a single honest signer remains, this signer necessarily acts as the last consistent signer, and our simulation strategy ensures that its blinded partial signature is both valid and indistinguishable from an honestly generated one.

Furthermore, this strategy does not allow the adversary to trigger $\mathcal{B}$ in wasting dLog oracle calls by sending inconsistent inputs: Each signer only answers the second-round oracle call a single time, since only once the honest first-round message will be accepted. For all signing oracle calls, $\mathcal{B}$ stores the inputs, s_{HS} , R_{HS} , s_i , and s'_i in the set SR_2 . In total, the signing simulation includes $2 \cdot |S \cap \text{HON}| \leq 2t$ dLog challenges and requires a single call to the dLog oracle.

Corruptions. When the adversary queries the corruption oracle on an index i , the reduction $\mathcal{B}$ first verifies that i is still an honest signer, i.e., $i \in \text{HON}$. If i has already been corrupted, the oracle aborts. Otherwise, the reduction $\mathcal{B}$ samples a fresh, uniformly random signing key $sk_i \leftarrow \$_{\mathbb{Z}_p}$. Then, let $K \leftarrow \{k : (k, i, \cdot) \in \text{SR}_1\}$ be the set of challenge indexes associated with party i , i.e., the indexes of the AOMDL challenges ever embedded into the first-round messages of party i . For each $k \in K$, $\mathcal{B}$ queries the discrete-logarithm oracle on input X_k (resp. X_{q_s+k}) to obtain the corresponding value d_i (resp. e_i) and stores them into SR_1 accordingly.

Next, $\mathcal{B}$ ensures that all previously generated second-round messages (i.e., the blinded partial signatures) remain consistent. For each stored record in SR_2 , it reconstructs the honest partial signature that the adversary should now be able to compute after corruption:

$$s_i'' \leftarrow sk_i \cdot c + d_i + b \cdot e_i,$$

where c is the corresponding Fiat–Shamir challenge, b is the nonce hash of the session (both stored in SR_2), and d_i, e_i are the values recovered from the first-round state. This value s_i'' represents the unblinded signature share that $\mathcal{A}$ would recompute upon receiving sk_i . Since after the corruption, the commitments (D_i, E_i) and sk_i are public, it is required that removing the blinding from the previously returned value s_i' yields exactly s_i'' . Finally, $\mathcal{B}$ adjusts the blinding values so that this consistency holds: the previously output (blinded) signature share s_i' must equal the sum of s_i'' and the combined blinding contributions from all other signers. We now describe the reprogramming step that restores consistency for the blinded partial signatures.

First, $\mathcal{B}$ computes the total correction term required to make the previously output blinded signature share s_i' consistent with the newly computed unblinded value s_i'' :

$$\delta_i \leftarrow s_i' - s_i'', \quad (2)$$

which represents the sum of all pairwise blinding contributions expected for signer i . Next, $\mathcal{B}$ determines the portion of this blinding that is already known to the adversary, namely the contributions corresponding to corrupted signers (whose pairwise keys are known):

$$B_i^{s, \text{CORR}} \leftarrow \sum_{j \neq i \in \text{CORR} \cap S} H_s(k_{i,j}, \text{sid}) \cdot \Delta_{i,j}.$$

For each remaining honest signer $j \in S \cap \text{HON}$, except for one designated last honest party j^* , $\mathcal{B}$ samples the corresponding blinding terms uniformly at random:

$$B_{i,j}^s = H_s(k_{i,j}, \text{sid}) \leftarrow \$_{\mathbb{Z}_p}.$$

Since we adjust this blinding retroactively and all at once, we can use for j^* simply the highest honest index in the signing set S . If i itself was the last remaining honest signer, we can skip this programming step, since the adversary already knows all blinding factors, hence we made the output consistent in a previous step. If this is not the case, i.e. prior to corruption at least two honest signers remained in this signer set, $\mathcal{B}$ reprograms the random oracle for the remaining pairwise key k_{i,j^*} so that the combined blinding sums to δ_i :

$$B_{i,j^*}^s = H_s(k_{i,j^*}, \text{sid}) = \left(\delta_i - \sum_{j \neq i \neq j^* \in S} B_{i,j}^s \cdot \Delta_{i,j} \right) \cdot \Delta_{i,j^*}. \quad (3)$$

Note that for making the last honest signer's blinding consistent, we use all other remaining blindings for the signing set, and not only the corrupted ones. Combining the unblinded partial signature s_i'' and all blindings,

we now have

$$\begin{aligned}
s_i'' + \sum_{j \neq i \in S} B_{i,j}^s &= s_i'' + \sum_{j \neq i \in S} B_{i,j}^s \cdot \Delta_{i,j} \\
&= s_i'' + \sum_{j \neq i \neq j^* \in S} B_{i,j}^s \cdot \Delta_{i,j} + B_{i,j^*}^s \cdot \Delta_{i,j^*} \\
&= s_i'' + \sum_{j \neq i \neq j^* \in S} B_{i,j}^s \cdot \Delta_{i,j} + \left(\delta_i - \sum_{j \neq i \neq j^* \in S} B_{i,j}^s \cdot \Delta_{i,j} \right) \cdot (\Delta_{i,j^*})^2 \\
&= s_i'' + \sum_{j \neq i \neq j^* \in S} B_{i,j}^s \cdot \Delta_{i,j} + \delta_i - \sum_{j \neq i \neq j^* \in S} B_{i,j}^s \cdot \Delta_{i,j} \\
&= s_i'' + \delta_i,
\end{aligned}$$

as required by Equation (2). Hence, we made our uniformly random signing consistent with the previously output signature shares.

Since we only do this programming, if at least one pairwise key k_{i,j^*} remains unknown to the adversary, and the first-round protocol messages are guaranteed to be unique for each signer ($\mathcal{B}$ would abort if the honest signer's nonce reused in $\{\text{pm}_{1,i}\}_{i \in S}$ and hence also sid were to be reused), the corresponding random-oracle input has never been queried before. Hence, this reprogramming step is consistent with the random-oracle model and indistinguishable from a freshly sampled response. It is also worth mentioning that making these outputs consistent is independent of whether the inputs to the sign-round two oracle were consistent or inconsistent, since this just represents the local view of a signer.

Eventually, the adversary $\mathcal{A}$ outputs a signature forgery $(R^*, s^*), m^*$, for which we have $c^* = \text{H}_{\text{sig}}(pk || R^* || m^*)$. In addition, $\mathcal{A}$ outputs a representation $(\alpha_{pk}, \alpha_g, \{(\beta_i, \gamma_i)\}_{1 \leq i \leq q_s})$ of R^* w.r.t. the prior output group elements $pk, g, \{(D_i, E_i)\}_{1 \leq i \leq q_s}$, such that

$$R^* = pk^{\alpha_{pk}} \cdot g^{\alpha_g} \cdot \prod_{1 \leq i \leq q_s} D_i^{\beta_i} \cdot \prod_{1 \leq i \leq q_s} E_i^{\gamma_i}$$

for $\alpha_{pk}, \alpha_g, \beta_i, \gamma_i \in \mathbb{Z}_p$.

Breaking the AOMDL assumption. It remains to show that we can solve the AOMDL problem whenever the adversary outputs a valid forgery, except with negligible probability. To make this connection precise, we frame the entire reduction as maintaining and solving a system of linear equations over unknowns from $\mathbb{Z}_p$.

Whenever the reduction embeds an AOMDL challenge element in the simulation, it adds an unknown to the system of equations, because the discrete logarithm of this element is unknown. Whenever the reduction calls the dLog oracle, a new linear equation, and hence, a new constraint, is added to the system of equations. In this view,

- every first-round signing oracle call corresponds to adding two unknowns (d_i, e_i) to the system
- every second-round oracle call, that relates to the last honest signer, corresponds to adding a new equation to the system;
- during corruption, each dLog oracle query reveals the value of one unknown, and hence removes this value from the set of unknowns.

The reduction succeeds if, at the time of the forgery, it has gathered enough linearly independent equations to solve for all remaining unknowns while having made at most $2 \cdot q_s$ many dLog oracle calls. We show that this is the case with overwhelming probability.

Init. If the adversary never queries the first-round signing oracle ($q_s = 0$), the only existing dLog challenge is pk . Since there is also no second-round signing oracle call, the reduction never needs to query the dLog oracle to attain consistency. When the adversary outputs a forgery (R^*, s^*) , it also provides an algebraic representation (α_{pk}, α_g) of R^* that can only depend on pk and g , since the reduction never outputs any other group element. The signature forgery is valid, therefore it holds that $pk^{c^*} \cdot R^* = g^{s^*}$. Therefore, we have

$$(c^* + \alpha_{pk} \alpha_g) \cdot \begin{pmatrix} sk \\ 1 \end{pmatrix} = s^*$$

for known α_{pk}, α_g , and c^* . This yields a unique solution for sk , because if $c^* = -\alpha_{pk}$, our solution would have already aborted as described before. Our reduction never queried the dLog oracle and, hence, breaks the AOMDL assumption.

First-Round Signing Oracles. For each first-round oracle call, our system of linear equations obtains two unknowns. For the k -th oracle query, these are (d_k, e_k) , which are the discrete logarithms to the challenges (D_k, E_k) . If the adversary outputs its forgery after only first-round oracle calls, our linear system has the form

$$(c^* + \alpha_{pk} \beta_1 \cdots \beta_{q_s} \gamma_1 \cdots \gamma_{q_s} \alpha_g) \cdot \begin{pmatrix} sk \\ d_1 \\ \vdots \\ d_{q_s} \\ e_1 \\ \vdots \\ e_{q_s} \\ 1 \end{pmatrix} = (s^*)$$

for known $c^*, (\beta_k, \gamma_k), \alpha_{pk}, \alpha_g$. If the adversary does not query q_s many first-round oracle calls, we set the corresponding values β_k, γ_k to zero. Since the adversary did not call the second-round signing oracle, the reduction was not required to query the dLog oracle to simulate the last consistent signer. Hence, it can solve the dLog of $2 \cdot q_s$ elements using the dLog oracle. Furthermore, $c^* + \alpha_{pk}$ is non-zero. Therefore, the reduction solves all d_k, e_k using the dLog oracle, and sk using the resulting linear system.

Second-Round Signing Oracles. Let $d_i^{(j)}$ and $e_i^{(j)}$ denote the discrete logarithms of the j -th pair of nonces received by the adversary, for the i -th session sid_i , when interacting with **SignRound**₁.

When the adversary queries the second-round signing oracle for the last-honest-signer, our reduction uses a dLog oracle call to compute the partial signature for this last-honest signer. At the same time, this partial signature of the last honest signer adds an equation to our system of linear equations. Recall that for the last-honest-signer to be consistent, our reduction computes the honest parties' combined share of the signature as $s_{\text{HON},i} = sk_{\text{HON},i} \cdot c_i + \sum_{j \in S \cap \text{HON}} d_i^{(j)} + e_i^{(j)} \cdot b_i$. When t honest parties are in the i -th session, this adds a constraint of the form

$$(c_i \ 1 \ \dots \ 1 \ b_i \ \dots \ b_i) \cdot \begin{pmatrix} sk \\ d_i^{(1)} \\ \vdots \\ d_i^{(t)} \\ e_i^{(1)} \\ \vdots \\ e_i^{(t)} \end{pmatrix} = (z_i),$$

to our system of linear equations. The challenges c_i and b_i are outputs of the random oracle as described in the simulation, and z_i is computed from the dLog oracle call. The above constraint can be adapted when the number of honest parties is not t (it must be at least $t - t_c$).

To ease readability, we will omit the vector containing the unknown values and solutions from this point forward, focusing solely on the coefficient matrix. Furthermore, we drop the last column, which refers to the known discrete logarithm α_g , and therefore does not refer to an unknown. After m last-consistent-signer queries, we can combine the constraints into a joined block-diagonal matrix. We provide an example with $m = 2, t = 3$

$$\begin{pmatrix} c_1 & 1 & 1 & 1 & 0 & 0 & 0 & b_1 & b_1 & b_1 & 0 & 0 & 0 \\ c_2 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & b_2 & b_2 & b_2 \end{pmatrix}.$$

We observe that these rows are linearly independent.

Challenge Random-Oracle Call. At some point, the adversary queries the signing random oracle on input (pk, R^*, m^*) and receives the value c^* that will later appear in the forgery. Together with this query, the adversary (or, if needed, the challenger) provides a representation of R^* as a linear combination of the group elements known so far, namely pk, g , and all D_i and E_i . This representation fixes the coefficients of the corresponding linear equation, but it does not yet yield a complete constraint since the solution s^* remains unknown until the adversary outputs its forgery. If the adversary never makes this query, the verifier's check performed by the challenger when validating the forgery serves as the relevant random-oracle call, leading to the same outcome for the reduction. At the moment of the challenge random-oracle query, the coefficient matrix takes the form

$$\begin{pmatrix} c_1 & 1 & 1 & 1 & 0 & 0 & 0 & b_1 & b_1 & b_1 & 0 & 0 & 0 \\ c_2 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & b_2 & b_2 & b_2 \\ c^* + \alpha_{pk} & \beta_1^{(1)} & \beta_1^{(2)} & \beta_1^{(3)} & \beta_2^{(1)} & \beta_2^{(2)} & \beta_2^{(3)} & \gamma_1^{(1)} & \gamma_1^{(2)} & \gamma_1^{(3)} & \gamma_2^{(1)} & \gamma_2^{(2)} & \gamma_2^{(3)} \end{pmatrix}.$$

For readability, we omit the final column corresponding to the known discrete logarithm α_g , which is not an unknown. Let $\beta_i^{(j)}$ and $\gamma_i^{(j)}$ denote the coefficients of $d_i^{(j)}$ and $e_i^{(j)}$, respectively; these coefficients correspond to exponents in the algebraic representation of group elements. If, at the time of the query, the adversary has not yet learned certain challenge elements or omits them from its representation, the corresponding coefficients are zero. We refer to this matrix as the *challenge matrix*, which will play a central role in the subsequent analysis. In the actual simulation, there would be many additional rows, one for each query to the signing random oracle H_{sig} on inputs (pk, R, m) together with the corresponding representation of R . Only after the adversary outputs a forgery (R^*, s^*, m^*) does the reduction retrospectively identify the *challenge* random oracle call as the first query to $H_{\text{sig}}(pk, R^*, m^*)$ that returned c^* . For exposition, we display only the row corresponding to this challenge call and omit all other irrelevant oracle queries. This is without loss of generality, as the reduction only derives information from the challenge random oracle call after the forgery has occurred, and the omitted rows do not influence our reasoning.

Corrupting a signer. When the adversary corrupts a signer, we follow our simulation strategy and solve all $D_i^{(j)}$ and $E_i^{(j)}$ that this signer has output in any prior first-round signing query. Since the corresponding discrete logarithms $d_i^{(j)}$ and $e_i^{(j)}$ are now known, we can remove them from the set of unknowns in our system of equations. We rewrite the corresponding equations by setting the coefficients of the $d_i^{(j)}$ s and $e_i^{(j)}$ s to zero, and updating the solutions of these equations accordingly. As an example, we depict the result of the corruption of the third party of the first signing session, resulting in

$$\begin{pmatrix} c_1 & 1 & 1 & 0 & 0 & 0 & 0 & b_1 & b_1 & 0 & 0 & 0 & 0 \\ c_2 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & b_2 & b_2 & b_2 \\ c^* + \alpha_{pk} & \beta_1^{(1)} & \beta_1^{(2)} & 0 & \beta_2^{(1)} & \beta_2^{(2)} & \beta_2^{(3)} & \gamma_1^{(1)} & \gamma_1^{(2)} & 0 & \gamma_2^{(1)} & \gamma_2^{(2)} & \gamma_2^{(3)} \end{pmatrix}.$$

We note that this strategy also works if the adversary corrupts parties prior to making the challenge oracle call, as we can simply perform the zeroing afterward.

The forgery matrix. Eventually, $\mathcal{A}$ outputs its forgery and cannot corrupt any party thereafter. In our matrix representation, this forgery corresponds to the reduction learning a solution for the challenge row (i.e., the row containing the challenge RO-query representation) of the challenge matrix. We refer to the matrix at the time of the forgery as the *forgery matrix*. If m last-consistent-signer sessions were queried before the forgery, the resulting forgery matrix has $m + 1$ rows. This dimensionality arises from the m consistency conditions provided by the last-consistent-signer constraints, plus the one additional constraint yielded by the valid forgery itself. The matrix includes one column for every honest signer in every signing session, resulting in a potentially higher number of unknowns than the number of constraints. In the remainder of the proof, we transform the $(m + 1) \times x$ *forgery matrix* (with $x \geq m + 1$) into a $(m + 1) \times (m + 1)$ *final matrix* of full rank such that we can solve for all remaining unknowns using standard techniques like Gaussian elimination. To see that we have enough leftover DLog oracle calls to remove the additional columns, note that for every challenge element except the public key, each DLog oracle call either (i) adds a row to the matrix, or (ii) removes an unknown and thus removes a column from the system of equations. As a result, the matrix ends up with the same number of rows and columns. The extra column for the public key is balanced by the additional row obtained from the representation that the adversary provides with the forgery. We give an overview of the challenge embedding and dLog oracle call progression in Table 2.

Table 2. Counting unknowns, equations, and dLog oracle queries in the reduction. The number $\text{OSignRound}_1(i)$ denotes the number of first-round signing calls for signer i .

Event / Oracle Call	New Unknowns	New Constraints	dLog Queries
Key generation	1 (sk)	0	0
First-round signing oracle	2 (d_i, e_i)	0	0
Second-round signing (non-last)	0	0	0
Second-round signing (last-hon-sig)	0	1	1
Corruption of a signer	$-2 \cdot \text{OSignRound}_1(i)$	0	$2 \cdot \text{OSignRound}_1(i)$
Forgery (final signature)	0	1	0

The only places we use dLog queries so far are during last-consistent-signer oracle calls and during corruption. During last-consistent-signer oracle calls, we use a dLog call to add a row to the matrix that is always linearly independent from all other last-consistent-signer rows, because of their block-diagonal structure. During corruption, we use dLog calls to remove columns. This never removes a row because there is always at least one honest signer left per signing session. In other words, at the point the adversary outputs its forgery, we can solve all but one embedded unknown in each last-consistent-signer session and all remaining challenge elements not used in a last-consistent-signer session.

Constraints on the forgery matrix. We will use the remaining dLog oracle calls to remove columns by solving individual challenge elements, similar to how we handle corruptions. To ensure that we end up with a matrix of full rank, we distinguish between different scenarios to decide which columns to remove. We begin with two cases, where the full rank of the final matrix follows directly from the structure of the forgery matrix. For each case, we provide examples with $m = 2$ for better readability, but the results hold for arbitrary m . We defer the general matrices to Appendix A.

1. $\exists(i, j) : \gamma_i^{(j)} \neq b_i \cdot \beta_i^{(j)}$: For all sessions $\ell \neq i$, the reduction solves all $e_\ell^{(\cdot)}$ and all but one $d_\ell^{(\cdot)}$. For the session i , the reduction solves all $e_i^{(\cdot)}, d_i^{(\cdot)}$ except for $e_i^{(j)}, d_i^{(j)}$. The reduction uses the remaining dLog oracle call of session i to solve for sk . For $i = 1$, the resulting matrix has the form

$$\begin{pmatrix} 1 & 0 & b_1 \\ 0 & 1 & 0 \\ \beta_1 & \beta_2 & \gamma_1 \end{pmatrix},$$

which has full rank since $\gamma_1 \neq b_1 \cdot \beta_1$, and hence $(\beta_1, \gamma_1) \neq (0, 0)$.

2. $\exists i : \exists j \neq k : \beta_i^{(j)} \neq \beta_i^{(k)}$: For all sessions $\ell \neq i$, the reduction solves all $e_\ell^{(\cdot)}$ and all but one $d_\ell^{(\cdot)}$. For the session i , the reduction solves all $e_i^{(\cdot)}$ and all $d_i^{(\cdot)}$ except for $d_i^{(j)}$ and $d_i^{(k)}$. With the leftover dLog oracle call for session i , the reduction solves for sk . For $i = 1$, the resulting matrix has the form

$$\begin{pmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \\ \beta_1^{(j)} & \beta_1^{(k)} & \beta_2 \end{pmatrix},$$

which has full rank since $\beta_1^{(j)} \neq \beta_1^{(k)}$.

Now we know that within a session, all β_i 's are the same, and for all γ_i 's it must hold that $\gamma_i = b_i \cdot \beta_i$. Otherwise, we say the adversary provides a forgery matrix with trivial non-linearity. By this, we mean that our reduction can solve the AOMDL problem using the above-described strategies. Based on these constraints, we can simplify the challenge matrix to be represented as

$$\begin{pmatrix} c_1 & 1 & 1 & 0 & 0 & 0 & 0 & b_1 & b_1 & 0 & 0 & 0 \\ c_2 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & b_2 & b_2 & b_2 \\ c^* + \alpha_{pk} & \beta_1 & \beta_1 & 0 & \beta_2 & \beta_2 & \beta_2 & b_1 \cdot \beta_1 & b_1 \cdot \beta_1 & b_2 \cdot \beta_2 & b_2 \cdot \beta_2 & b_2 \cdot \beta_2 \end{pmatrix}.$$

For all sessions ℓ , the reduction solves all $e_\ell^{(\cdot)}$ and all but one $d_\ell^{(\cdot)}$. The resulting matrix, which we call the *final matrix*, has the form

$$\begin{pmatrix} c_1 & 1 & 0 \\ c_2 & 0 & 1 \\ c^* + \alpha_{pk} & \beta_1 & \beta_2 \end{pmatrix}.$$

It remains to show that the final matrix has full rank. Due to the $m \times m$ identity submatrix in the top-left corner, it suffices to ensure that the last row is not in the span of the first m ones. This holds if

$$c^* \neq \sum_{i=1}^m \beta_i \cdot c_i - \alpha_{pk}.$$

The values β_i , c_i , and α_{pk} are already fixed when c^* is sampled uniformly at random with overwhelming probability. The values c_i are fixed when the b_i values are fixed, due to our random-oracle programming strategy of H_{non} . Since we are not in Case 1, the adversary must know the b_i values used by honest signers. The only way to achieve this *without* querying H_{non} prior to the challenge RO call is by correctly guessing. Such a guessing succeeds with probability at most q_{sig}/p , as each potential challenge RO call has probability of $1/p$ in guessing b_i correct, and there are at most q_{sig} possible challenge RO calls.

Furthermore, the algebraic representation of R^* determines all $\beta_i^{(j)}$ and α_{pk} . The last degree of freedom for the adversary is to pick one β_i out of the possible $\beta_i^{(j)}$ for each session i . We show that this transformation admits at most $t/(t - t_c)$ possible choices in total. Since there are at most q_{sig} different challenge matrices, this increases the adversary's success probability in failing the reduction by $\frac{q_{\text{sig}} \cdot t}{(t - t_c) p}$.

Possible Matrix Transformations. As a warmup for formally proving that a single challenge matrix results in at most $t/(t - t_c)$ possible forgery matrices without imposing a trivial non-linearity (i.e., violating Case 2), we provide an example with $t = 3$, $t_c = 2$, $n = 5$, and three signing sessions. For our argument, we focus on the challenge vector, i.e., the row of the challenge matrix that contains the adversarially chosen representation. For readability, we represent each signing session as a subvector of size n and set the value of non-participating, or corrupted, signers to 0. For our example, we assume the following:

- Signing session 1: the adversary used the same β_1 for parties 1 and 2, and a different one for party 3.
- Signing session 2: the adversary used the same β_2 for parties 3 and 4, and a different one for party 2.

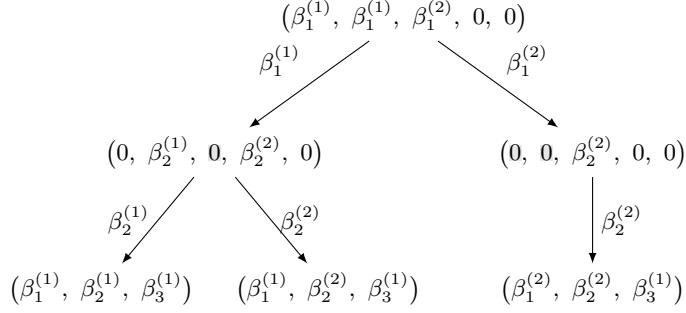


Fig. 7. Adversarial branching by selecting candidate values. We omit the third level, as in our example, this level only exposes a single candidate value $\beta_3^{(1)}$.

- Signing session 3: the adversary used the same β_3 for parties 3, 4, and 5.

This results in the vector $\mathbf{v}$ defined as

$$\mathbf{v} = ((\beta_1^{(1)}, \beta_1^{(1)}, \beta_1^{(2)}, 0, 0), (0, \beta_2^{(1)}, \beta_2^{(2)}, \beta_2^{(2)}, 0), (0, 0, \beta_3^{(1)}, \beta_3^{(1)}, \beta_3^{(1)})).$$

As discussed above, the adversary has to select a single value β_i for each signing session. We denote the values $\beta_i^{(j)}$ as the candidate value for session i . We will examine all possible combinations of candidate values in the final matrix by iterating over all sessions and all choices of candidate values. For the first signing session $(\beta_1^{(1)}, \beta_1^{(1)}, \beta_1^{(2)}, 0, 0)$, there are two choices $\beta_1^{(1)}$ or $\beta_1^{(2)}$. If the adversary wants to pick $\beta_1^{(1)}$, it needs to corrupt party three (We discuss the choice of $\beta_1^{(2)}$ below). After corrupting party three, the updated vector $\mathbf{v}'$ has the form

$$\mathbf{v}' = ((\beta_1^{(1)}, \beta_1^{(1)}, 0, 0, 0), (0, \beta_2^{(1)}, 0, \beta_2^{(2)}, 0), (0, 0, 0, \beta_3^{(1)}, \beta_3^{(1)})),$$

and the adversary has one corruption left. The second signing session subvector has been modified due to the corruption introduced in the first choice and now has the form $(0, \beta_2^{(1)}, 0, \beta_2^{(2)}, 0)$. In this updated second session, the adversary can either choose $\beta_2^{(1)}$ by corrupting party four, or choose the value $\beta_2^{(2)}$ by corrupting party two. In either case, the adversary consumes the last corruption. Now, the adversary has no freedom of choice, so in all subsequent sessions, there can be only a single candidate left. Otherwise, the condition of the forgery matrix would be violated. This is the case in our example, so this choice of the adversary provides the possible solutions

$$\{(\beta_1^{(1)}, \beta_2^{(1)}, \beta_3^{(1)}), (\beta_1^{(1)}, \beta_2^{(2)}, \beta_3^{(1)})\}.$$

To select the second possible value in the first set, the adversary needs to corrupt parties one and two. The remaining vector of updated sessions then has the form

$$\mathbf{v}'' = ((0, 0, \beta_1^{(2)}, 0, 0), (0, 0, \beta_2^{(2)}, \beta_2^{(2)}, 0), (0, 0, \beta_3^{(1)}, \beta_3^{(1)}, \beta_3^{(1)}))$$

and the adversary has no corruptions left. Again, because of the condition on the forgery matrix, it is ensured that once the adversary has no corruptions left, the subsequent sessions can only have a single candidate value each. Therefore, the set of possible choices expands to

$$\{(\beta_1^{(1)}, \beta_2^{(1)}, \beta_3^{(1)}), (\beta_1^{(1)}, \beta_2^{(2)}, \beta_3^{(1)}), (\beta_1^{(2)}, \beta_2^{(2)}, \beta_3^{(1)})\}.$$

There are 3 possible choice combinations, which is less than or equal to $t/(t - t_c)$. We depict the adversary's choices in a tree diagram in Figure 7.

With this intuition in mind, we now prove that our claim holds. I.e., we show that if the adversary does not impose a trivial non-linearity, there are at most $t/(t - t_c)$ many different forgery matrices. We

do this in two steps. First, we present a *local bound*: the maximum number of different candidate values within a (possibly updated) signing session depends only on the number of remaining honest signers t_i and the remaining corruption capability $c_{\max,i}$, which initially equals t_c . Whenever a corruption occurs, $c_{\max,i}$ is reduced by the number of consumed corruptions. Then, we use this local bound to derive a *global bound*. As the adversary progresses through the signing sessions, each choice of a candidate value consumes part of its remaining corruption capability, thereby reducing the freedom for subsequent sessions. Formally, we show that the sum of the maximum number of choices for all successor sessions never exceeds the maximum number of choices for the current session. We define the maximum number of choices independent of the actual values $\beta_i^{(j)}$, but only with respect to the remaining corruption capabilities. This invariant ensures that, although later sessions may present multiple branching options, the total potential is conserved; combined with the local bound, this yields the desired global limit on the number of distinct final vectors.

Lemma 1 (Local Bound: Maximum Candidates per Session). *Let i be a session containing t_i honest parties, and let $\mathbf{v}_i = (\beta_1^{(1)}, \beta_1^{(2)}, \dots, \beta_i^{(t_i)})$ be the associated session vector. An adversary, that can corrupt up to $c_{\max,i} < t_i$ parties, can provide up to*

$$\nu_i \leq \left\lfloor \frac{t_i}{t_i - c_{\max,i}} \right\rfloor$$

different candidates in a signing session i without imposing a trivial non-linearity.

Proof (Lemma 1). Let ν_i be the number of final candidates. For a candidate value $\beta_i^{(j)}$ with a multiplicity $m_i^{(j)}$ to be a final candidate, the adversary must corrupt all honest parties holding a *different* value. With a budget of $c_{\max,i}$, this is possible only if $\beta_i^{(j)}$ appears $m_i^{(j)} \geq t_i - c_{\max,i}$ times in $\mathbf{v}_i$. Hence the ν_i candidate values together occupy at least $\nu_i \cdot (t_i - c_{\max,i})$ positions among the t_i honest parties. Since we have at most t_i honest parties, it holds $\nu_i \cdot (t_i - c_{\max,i}) \leq t_i$, and therefore,

$$\nu_i \leq \frac{t_i}{t_i - c_{\max,i}}.$$

Since ν_i is an integer, $\nu_i \leq \lfloor t_i / (t_i - c_{\max,i}) \rfloor$, as claimed.

Lemma 2 (Global Bound: Preserving the Number of Candidates). *Let signing session i contain t_i honest parties, and let the adversary have remaining corruption capacity $c_{\max,i} < t_i$. Denote by ν_i the current number of candidate values in session i (c.f. Lemma 1), by $\hat{\nu}_i$ the maximal number of feasible candidates for the given corruption capacity $c_{\max,i}$, and by $\hat{\nu}_{i,j}$ the corresponding maximum for the successor session that follows after selecting candidate j . Then,*

$$\sum_{j=1}^{\nu_i} \hat{\nu}_{i,j} \leq \hat{\nu}_i.$$

Proof (Lemma 2). From Lemma 1, the number of possible candidates in any session with parameters $(t_i, c_{\max,i})$ satisfies

$$\nu_i \leq \left\lfloor \frac{t_i}{t_i - c_{\max,i}} \right\rfloor. \quad (4)$$

We must assume $c_{\max,i} < t_i$, and $t_c < t$. Therefore, ν_i is strictly increasing in $c_{\max,i}$ and reaches its maximum if $c_{\max,i} = t_i - (t - t_c)$, i.e., if the maximal possible corruption allows to corrupt all but $t - t_c$ out of the t_i honest parties. Substituting this in Equation (4) gives

$$\hat{\nu}_i = \left\lfloor \frac{c_{\max,i} + t - t_c}{t - t_c} \right\rfloor \quad \text{and} \quad \hat{\nu}_{i,j} = \left\lfloor \frac{c_{\max,i,j} + t - t_c}{t - t_c} \right\rfloor. \quad (5)$$

We observe that the denominator is fixed for all i ; therefore, we focus on the numerator first. For each feasible candidate $\beta_i^{(j)}$, let m_j be its multiplicity among the honest parties. Selecting $\beta_i^{(j)}$ requires corrupting

all other honest parties, i.e., $\delta_j = t_i - m_j$, leaving $c_{\max,i,j} = c_{\max,i} - \delta_j \geq 0$. We first show that the total remaining corruption capacity across all successors never exceeds the current one:

$$\sum_{j=1}^{\nu_i} (c_{\max,i,j} + (t - t_c)) = \sum_{j=1}^{\nu_i} (c_{\max,i} - \delta_j + (t - t_c)) = \nu_i \cdot (c_{\max,i} + (t - t_c)) - \sum_{j=1}^{\nu_i} \delta_j. \quad (6)$$

Substituting $\delta_j = t_i - m_j$ and using $\sum_{j=1}^{\nu_i} m_j = t_i$ yields

$$\sum_{j=1}^{\nu_i} \delta_j = \sum_{j=1}^{\nu_i} (t_i - m_j) = \nu_i t_i - \sum_{j=1}^{\nu_i} m_j = \nu_i t_i - t_i = (\nu_i - 1)t_i, \quad (7)$$

and hence we can substitute Equation (7) in Equation (6), obtaining

$$\sum_{j=1}^{\nu_i} c_{\max,i,j} + (t - t_c) = \nu_i \cdot (c_{\max,i} + (t - t_c)) - (\nu_i - 1)t_i. \quad (8)$$

We now want to substitute t_i , and to do this, we use the observation above stating that $t_i \geq c_{\max,i} + (t - t_c)$. We perform this substitution in Equation (8) resulting in

$$\sum_{j=1}^{\nu_i} c_{\max,i,j} + (t - t_c) \leq \nu_i (c_{\max,i} + (t - t_c)) - (\nu_i - 1)(c_{\max,i} + (t - t_c)) = c_{\max,i} + (t - t_c). \quad (9)$$

Applying the formula of the maximum number of candidate values given in Equation (5), using the subadditivity of the floor function, and finally applying the inequality of Equation (9) results in

$$\sum_{j=1}^{\nu_i} \hat{\nu}_{i,j} = \sum_{j=1}^{\nu_i} \left\lfloor \frac{c_{\max,i,j} + (t - t_c)}{t - t_c} \right\rfloor \leq \left\lfloor \sum_{j=1}^{\nu_i} \frac{c_{\max,i,j} + (t - t_c)}{t - t_c} \right\rfloor \leq \left\lfloor \frac{c_{\max,i} + (t - t_c)}{t - t_c} \right\rfloor = \hat{\nu}_i,$$

and our lemma holds.

Putting everything together, our claim holds, since $\hat{\nu}_0 \leq t/(t - t_c)$, therefore there also exist at most $t/(t - t_c)$ forgery matrices without imposing a trivial non-linearity.

This concludes our Proof. We have demonstrated that if the reduction does not abort, it outputs $2 \cdot q_s + 1$ solutions to the AOMDL problem while querying the dLog oracle at most $2 \cdot q_s$ times. Additionally, we have demonstrated that the probability of aborting the reduction is negligible. This demonstrates the unforgeability of our scheme against fully adaptive adversaries.

Acknowledgements

We thank Kaoru Takemure and Nadav Kohen for their helpful comments on an earlier version of our proof. They pointed out that our analysis omitted a step needed to fully account for ROS attacks in our simulation and that our probability argument did not capture the adversary's multiple options for transforming the challenge matrix into a forgery matrix. We addressed these issues, which increases the reduction's failure probability by a factor of $\frac{t}{t - t_c} \leq t$. We also thank the authors of [2] for independently identifying the latter observation and bringing it to our attention.

References

- [1] Damiano Abram, Ariel Nof, Claudio Orlandi, Peter Scholl, and Omer Shlomovits. “Low-Bandwidth Threshold ECDSA via Pseudorandom Correlation Generators”. In: *2022 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2022, pp. 2554–2572. DOI: [10.1109/SP46214.2022.9833559](https://doi.org/10.1109/SP46214.2022.9833559).
- [2] Renas Bacho, Yanbo Chen, Julian Loss, Stefano Tessaro, and Chenzhi Zhu. *Adaptively Secure Partially Non-Interactive Threshold Schnorr Signatures in the AGM*. Cryptology ePrint Archive, Paper 2025/1953. 2025. URL: <https://eprint.iacr.org/2025/1953>.
- [3] Renas Bacho, Sourav Das, Julian Loss, and Ling Ren. “Adaptively Secure Three-Round Threshold Schnorr Signatures from DDH”. In: *CRYPTO 2025, Part VI*. Ed. by Yael Tauman Kalai and Seny F. Kamara. Vol. 16005. LNCS. Springer, Cham, Aug. 2025, pp. 390–422. DOI: [10.1007/978-3-032-01887-8_13](https://doi.org/10.1007/978-3-032-01887-8_13).
- [4] Renas Bacho, Sourav Das, Julian Loss, and Ling Ren. “Glacius: Threshold Schnorr Signatures from DDH with Full Adaptive Security”. In: *EUROCRYPT 2025, Part II*. Ed. by Serge Fehr and Pierre-Alain Fouque. Vol. 15602. LNCS. Springer, Cham, May 2025, pp. 304–334. DOI: [10.1007/978-3-031-91124-8_11](https://doi.org/10.1007/978-3-031-91124-8_11).
- [5] Mihir Bellare, Elizabeth C. Crites, Chelsea Komlo, Mary Maller, Stefano Tessaro, and Chenzhi Zhu. “Better than Advertised Security for Non-interactive Threshold Signatures”. In: *CRYPTO 2022, Part IV*. Ed. by Yevgeniy Dodis and Thomas Shrimpton. Vol. 13510. LNCS. Springer, Cham, Aug. 2022, pp. 517–550. DOI: [10.1007/978-3-031-15985-5_18](https://doi.org/10.1007/978-3-031-15985-5_18).
- [6] Mihir Bellare and Gregory Neven. “Multi-signatures in the plain public-Key model and a general forking lemma”. In: *ACM CCS 2006*. Ed. by Ari Juels, Rebecca N. Wright, and Sabrina De Capitani di Vimercati. ACM Press, 2006, pp. 390–399. DOI: [10.1145/1180405.1180453](https://doi.org/10.1145/1180405.1180453).
- [7] Manuel Blum, Paul Feldman, and Silvio Micali. “Non-Interactive Zero-Knowledge and Its Applications (Extended Abstract)”. In: *20th ACM STOC*. ACM Press, May 1988, pp. 103–112. DOI: [10.1145/62212.62222](https://doi.org/10.1145/62212.62222).
- [8] Alexandra Boldyreva. “Threshold Signatures, Multisignatures and Blind Signatures Based on the Gap-Diffie-Hellman-Group Signature Scheme”. In: *PKC 2003*. Ed. by Yvo Desmedt. Vol. 2567. LNCS. Springer, Berlin, Heidelberg, Jan. 2003, pp. 31–46. DOI: [10.1007/3-540-36288-6_3](https://doi.org/10.1007/3-540-36288-6_3).
- [9] Luís T. A. N. Brandão and Michael Davidson. *Notes on Threshold EdDSA/Schnorr Signatures*. Tech. rep. NIST IR 8214B (Draft). Initial Public Draft. National Institute of Standards and Technology (NIST), 2022. DOI: [10.6028/NIST.IR.8214B.ipd](https://doi.org/10.6028/NIST.IR.8214B.ipd). URL: <https://doi.org/10.6028/NIST.IR.8214B.ipd>.
- [10] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. “Bulletproofs: Short Proofs for Confidential Transactions and More”. In: *2018 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2018, pp. 315–334. DOI: [10.1109/SP.2018.00020](https://doi.org/10.1109/SP.2018.00020).
- [11] Matteo Campanelli, Antonio Faonio, and Luigi Russo. “SNARKs for Virtual Machines Are Non-malleable”. In: *EUROCRYPT 2025, Part IV*. Ed. by Serge Fehr and Pierre-Alain Fouque. Vol. 15604. LNCS. Springer, Cham, May 2025, pp. 153–183. DOI: [10.1007/978-3-031-91134-7_6](https://doi.org/10.1007/978-3-031-91134-7_6).
- [12] Ran Canetti, Rosario Gennaro, Steven Goldfeder, Nikolaos Makriyannis, and Udi Peled. “UC Non-Interactive, Proactive, Threshold ECDSA with Identifiable Aborts”. In: *ACM CCS 2020*. Ed. by Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna. ACM Press, Nov. 2020, pp. 1769–1787. DOI: [10.1145/3372297.3423367](https://doi.org/10.1145/3372297.3423367).
- [13] Guilhem Castagnos, Dario Catalano, Fabien Laguillaumie, Federico Savasta, and Ida Tucker. “Bandwidth-Efficient Threshold EC-DSA”. In: *PKC 2020, Part II*. Ed. by Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas. Vol. 12111. LNCS. Springer, Cham, May 2020, pp. 266–296. DOI: [10.1007/978-3-030-45388-6_10](https://doi.org/10.1007/978-3-030-45388-6_10).
- [14] David Chaum and Torben P. Pedersen. “Wallet Databases with Observers”. In: *CRYPTO’92*. Ed. by Ernest F. Brickell. Vol. 740. LNCS. Springer, Berlin, Heidelberg, Aug. 1993, pp. 89–105. DOI: [10.1007/3-540-48071-4_7](https://doi.org/10.1007/3-540-48071-4_7).

- [15] Yanbo Chen. “Round-Efficient Adaptively Secure Threshold Signatures with Rewinding”. In: *IACR Communications in Cryptology* 2.2 (July 7, 2025). ISSN: 3006-5496. DOI: [10.62056/ah893z10k](https://doi.org/10.62056/ah893z10k).
- [16] Hien Chu, Paul Gerhart, Tim Ruffing, and Dominique Schröder. “Practical Schnorr Threshold Signatures Without the Algebraic Group Model”. In: *CRYPTO 2023, Part I*. Ed. by Helena Handschuh and Anna Lysyanskaya. Vol. 14081. LNCS. Springer, Cham, Aug. 2023, pp. 743–773. DOI: [10.1007/978-3-031-38557-5_24](https://doi.org/10.1007/978-3-031-38557-5_24).
- [17] Deirdre Connolly, Chelsea Komlo, Ian Goldberg, and Christopher A. Wood. *The Flexible Round-Optimized Schnorr Threshold (FROST) Protocol for Two-Round Schnorr Signatures*. RFC 9591. June 2024. DOI: [10.17487/RFC9591](https://doi.org/10.17487/RFC9591). URL: <https://www.rfc-editor.org/info/rfc9591>.
- [18] Elizabeth Crites, Chelsea Komlo, and Mary Maller. *How to Prove Schnorr Assuming Schnorr: Security of Multi- and Threshold Signatures*. Cryptology ePrint Archive, Report 2021/1375. 2021. URL: <https://eprint.iacr.org/2021/1375>.
- [19] Elizabeth Crites, Chelsea Komlo, and Mary Maller. *On the Adaptive Security of Key-Unique Threshold Signatures*. Cryptology ePrint Archive, Paper 2025/943. 2025. URL: <https://eprint.iacr.org/2025/943>.
- [20] Elizabeth C. Crites, Jonathan Katz, Chelsea Komlo, Stefano Tessaro, and Chenzhi Zhu. “On the Adaptive Security of FROST”. In: *CRYPTO 2025, Part VI*. Ed. by Yael Tauman Kalai and Seny F. Kamara. Vol. 16005. LNCS. Springer, Cham, Aug. 2025, pp. 480–511. DOI: [10.1007/978-3-032-01887-8_16](https://doi.org/10.1007/978-3-032-01887-8_16).
- [21] Elizabeth C. Crites, Chelsea Komlo, and Mary Maller. “Fully Adaptive Schnorr Threshold Signatures”. In: *CRYPTO 2023, Part I*. Ed. by Helena Handschuh and Anna Lysyanskaya. Vol. 14081. LNCS. Springer, Cham, Aug. 2023, pp. 678–709. DOI: [10.1007/978-3-031-38557-5_22](https://doi.org/10.1007/978-3-031-38557-5_22).
- [22] Elizabeth C. Crites and Alistair Stewart. “A Plausible Attack on the Adaptive Security of Threshold Schnorr Signatures”. In: *CRYPTO 2025, Part VI*. Ed. by Yael Tauman Kalai and Seny F. Kamara. Vol. 16005. LNCS. Springer, Cham, Aug. 2025, pp. 457–479. DOI: [10.1007/978-3-032-01887-8_15](https://doi.org/10.1007/978-3-032-01887-8_15).
- [23] Anders P. K. Dalskov, Claudio Orlandi, Marcel Keller, Kris Shrishak, and Haya Shulman. “Securing DNSSEC Keys via Threshold ECDSA from Generic MPC”. In: *ESORICS 2020, Part II*. Ed. by Liqun Chen, Ninghui Li, Kaitai Liang, and Steve A. Schneider. Vol. 12309. LNCS. Springer, Cham, Sept. 2020, pp. 654–673. DOI: [10.1007/978-3-030-59013-0_32](https://doi.org/10.1007/978-3-030-59013-0_32).
- [24] Ivan Damgård, Thomas Pelle Jakobsen, Jesper Buus Nielsen, Jakob Illeborg Pagter, and Michael Bækvang Østergaard. “Fast Threshold ECDSA with Honest Majority”. In: *SCN 20*. Ed. by Clemente Galdi and Vladimir Kolesnikov. Vol. 12238. LNCS. Springer, Cham, Sept. 2020, pp. 382–400. DOI: [10.1007/978-3-030-57990-6_19](https://doi.org/10.1007/978-3-030-57990-6_19).
- [25] Alfredo De Santis, Yvo Desmedt, Yair Frankel, and Moti Yung. “How to Share a Function Securely”. In: *26th ACM STOC*. ACM Press, May 1994, pp. 522–533. DOI: [10.1145/195058.195405](https://doi.org/10.1145/195058.195405).
- [26] Alfredo De Santis, Giovanni Di Crescenzo, Rafail Ostrovsky, Giuseppe Persiano, and Amit Sahai. “Robust Non-interactive Zero Knowledge”. In: *CRYPTO 2001*. Ed. by Joe Kilian. Vol. 2139. LNCS. Springer, Berlin, Heidelberg, Aug. 2001, pp. 566–598. DOI: [10.1007/3-540-44647-8_33](https://doi.org/10.1007/3-540-44647-8_33).
- [27] Yvo Desmedt. “Society and Group Oriented Cryptography: A New Concept”. In: *CRYPTO’87*. Ed. by Carl Pomerance. Vol. 293. LNCS. Springer, Berlin, Heidelberg, Aug. 1988, pp. 120–127. DOI: [10.1007/3-540-48184-2_8](https://doi.org/10.1007/3-540-48184-2_8).
- [28] Yvo Desmedt and Yair Frankel. “Threshold Cryptosystems”. In: *CRYPTO’89*. Ed. by Gilles Brassard. Vol. 435. LNCS. Springer, New York, Aug. 1990, pp. 307–315. DOI: [10.1007/0-387-34805-0_28](https://doi.org/10.1007/0-387-34805-0_28).
- [29] Jack Doerner, Yashvanth Kondi, Eysa Lee, and abhi shelat. “Threshold ECDSA from ECDSA Assumptions: The Multiparty Case”. In: *2019 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2019, pp. 1051–1066. DOI: [10.1109/SP.2019.00024](https://doi.org/10.1109/SP.2019.00024).
- [30] Georg Fuchsbauer, Eike Kiltz, and Julian Loss. “The Algebraic Group Model and its Applications”. In: *CRYPTO 2018, Part II*. Ed. by Hovav Shacham and Alexandra Boldyreva. Vol. 10992. LNCS. Springer, Cham, Aug. 2018, pp. 33–62. DOI: [10.1007/978-3-319-96881-0_2](https://doi.org/10.1007/978-3-319-96881-0_2).

- [31] Adam Gągol, Jędrzej Kula, Damian Straszak, and Michał Świątek. *Threshold ECDSA for Decentralized Asset Custody*. Cryptology ePrint Archive, Report 2020/498. 2020. URL: <https://eprint.iacr.org/2020/498>.
- [32] Rosario Gennaro and Steven Goldfeder. “Fast Multiparty Threshold ECDSA with Fast Trustless Setup”. In: *ACM CCS 2018*. Ed. by David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang. ACM Press, Oct. 2018, pp. 1179–1194. DOI: [10.1145/3243734.3243859](https://doi.org/10.1145/3243734.3243859).
- [33] Rosario Gennaro, Steven Goldfeder, and Arvind Narayanan. “Threshold-Optimal DSA/ECDSA Signatures and an Application to Bitcoin Wallet Security”. In: *ACNS 2016*. Ed. by Mark Manulis, Ahmad-Reza Sadeghi, and Steve Schneider. Vol. 9696. LNCS. Springer, Cham, June 2016, pp. 156–174. DOI: [10.1007/978-3-319-39555-5_9](https://doi.org/10.1007/978-3-319-39555-5_9).
- [34] Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. “Robust Threshold DSS Signatures”. In: *EUROCRYPT’96*. Ed. by Ueli M. Maurer. Vol. 1070. LNCS. Springer, Berlin, Heidelberg, May 1996, pp. 354–371. DOI: [10.1007/3-540-68339-9_31](https://doi.org/10.1007/3-540-68339-9_31).
- [35] Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. “Secure Applications of Pedersen’s Distributed Key Generation Protocol”. In: *CT-RSA 2003*. Ed. by Marc Joye. Vol. 2612. LNCS. Springer, Berlin, Heidelberg, Apr. 2003, pp. 373–390. DOI: [10.1007/3-540-36563-X_26](https://doi.org/10.1007/3-540-36563-X_26).
- [36] Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. “Secure Distributed Key Generation for Discrete-Log Based Cryptosystems”. In: *Journal of Cryptology* 20.1 (Jan. 2007), pp. 51–83. DOI: [10.1007/s00145-006-0347-3](https://doi.org/10.1007/s00145-006-0347-3).
- [37] Rosario Gennaro, Tal Rabin, Stanislaw Jarecki, and Hugo Krawczyk. “Robust and Efficient Sharing of RSA Functions”. In: *Journal of Cryptology* 13.2 (Mar. 2000), pp. 273–300. DOI: [10.1007/s001459910011](https://doi.org/10.1007/s001459910011).
- [38] Craig Gentry and Daniel Wichs. “Separating succinct non-interactive arguments from all falsifiable assumptions”. In: *43rd ACM STOC*. Ed. by Lance Fortnow and Salil P. Vadhan. ACM Press, June 2011, pp. 99–108. DOI: [10.1145/1993636.1993651](https://doi.org/10.1145/1993636.1993651).
- [39] Paul Gerhart, Davide Li Calsi, Luigi Russo, and Dominique Schröder. *Bounded-Equivocable Pseudorandom Functions*. Cryptology ePrint Archive, Paper 2025/1894. 2025. URL: <https://eprint.iacr.org/2025/1894>.
- [40] Paul Gerhart, Davide Li Calsi, Luigi Russo, and Dominique Schröder. *Fully-Adaptive Two-Round Threshold Schnorr Signatures from DDH*. Cryptology ePrint Archive, Paper 2025/1478. 2025. URL: <https://eprint.iacr.org/2025/1478>.
- [41] Jens Groth and Mary Maller. “Snarky Signatures: Minimal Signatures of Knowledge from Simulation-Extractable SNARKs”. In: *CRYPTO 2017, Part II*. Ed. by Jonathan Katz and Hovav Shacham. Vol. 10402. LNCS. Springer, Cham, Aug. 2017, pp. 581–612. DOI: [10.1007/978-3-319-63715-0_20](https://doi.org/10.1007/978-3-319-63715-0_20).
- [42] Shuichi Katsumata, Michael Reichle, and Kaoru Takemure. “Adaptively Secure 5 Round Threshold Signatures from MLWE/MSIS and DL with Rewinding”. In: *CRYPTO 2024, Part VII*. Ed. by Leonid Reyzin and Douglas Stebila. Vol. 14926. LNCS. Springer, Cham, Aug. 2024, pp. 459–491. DOI: [10.1007/978-3-031-68394-7_15](https://doi.org/10.1007/978-3-031-68394-7_15).
- [43] Chelsea Komlo and Ian Goldberg. “FROST: Flexible Round-Optimized Schnorr Threshold Signatures”. In: *SAC 2020*. Ed. by Orr Dunkelman, Michael J. Jacobson Jr., and Colin O’Flynn. Vol. 12804. LNCS. Springer, Cham, Oct. 2020, pp. 34–65. DOI: [10.1007/978-3-030-81652-0_2](https://doi.org/10.1007/978-3-030-81652-0_2).
- [44] Yehuda Lindell and Ariel Nof. “Fast Secure Multiparty ECDSA with Practical Distributed Key Generation and Applications to Cryptocurrency Custody”. In: *ACM CCS 2018*. Ed. by David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang. ACM Press, Oct. 2018, pp. 1837–1854. DOI: [10.1145/3243734.3243788](https://doi.org/10.1145/3243734.3243788).
- [45] Ueli M. Maurer. “Abstract Models of Computation in Cryptography (Invited Paper)”. In: *10th IMA International Conference on Cryptography and Coding*. Ed. by Nigel P. Smart. Vol. 3796. LNCS. Springer, Berlin, Heidelberg, Dec. 2005, pp. 1–12. DOI: [10.1007/11586821_1](https://doi.org/10.1007/11586821_1).
- [46] Jonas Nick, Tim Ruffing, and Yannick Seurin. “MuSig2: Simple Two-Round Schnorr Multi-signatures”. In: *CRYPTO 2021, Part I*. Ed. by Tal Malkin and Chris Peikert. Vol. 12825. LNCS. Virtual Event: Springer, Cham, Aug. 2021, pp. 189–221. DOI: [10.1007/978-3-030-84242-0_8](https://doi.org/10.1007/978-3-030-84242-0_8).

- [47] Rafaël Del Pino, Shuichi Katsumata, Guilhem Niot, Michael Reichle, and Kaoru Takemure. “Unmasking TRaccoon: A Lattice-Based Threshold Signature with An Efficient Identifiable Abort Protocol”. In: *CRYPTO 2025, Part VI*. Ed. by Yael Tauman Kalai and Seny F. Kamara. Vol. 16005. LNCS. Springer, Cham, Aug. 2025, pp. 423–456. DOI: [10.1007/978-3-032-01887-8_14](https://doi.org/10.1007/978-3-032-01887-8_14).
- [48] Tim Ruffing, Viktoria Ronge, Elliott Jin, Jonas Schneider-Bensch, and Dominique Schröder. “ROAST: Robust Asynchronous Schnorr Threshold Signatures”. In: *ACM CCS 2022*. Ed. by Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi. ACM Press, Nov. 2022, pp. 2551–2564. DOI: [10.1145/3548606.3560583](https://doi.org/10.1145/3548606.3560583).
- [49] Claus-Peter Schnorr. “Efficient Identification and Signatures for Smart Cards”. In: *CRYPTO’89*. Ed. by Gilles Brassard. Vol. 435. LNCS. Springer, New York, Aug. 1990, pp. 239–252. DOI: [10.1007/0-387-34805-0_22](https://doi.org/10.1007/0-387-34805-0_22).
- [50] Victor Shoup. “Lower Bounds for Discrete Logarithms and Related Problems”. In: *EUROCRYPT’97*. Ed. by Walter Fumy. Vol. 1233. LNCS. Springer, Berlin, Heidelberg, May 1997, pp. 256–266. DOI: [10.1007/3-540-69053-0_18](https://doi.org/10.1007/3-540-69053-0_18).
- [51] Victor Shoup. “Practical Threshold Signatures”. In: *EUROCRYPT 2000*. Ed. by Bart Preneel. Vol. 1807. LNCS. Springer, Berlin, Heidelberg, May 2000, pp. 207–220. DOI: [10.1007/3-540-45539-6_15](https://doi.org/10.1007/3-540-45539-6_15).
- [52] Tsz Hon Yuen, Handong Cui, and Xiang Xie. “Compact Zero-Knowledge Proofs for Threshold ECDSA with Trustless Setup”. In: *PKC 2021, Part I*. Ed. by Juan Garay. Vol. 12710. LNCS. Springer, Cham, May 2021, pp. 481–511. DOI: [10.1007/978-3-030-75245-3_18](https://doi.org/10.1007/978-3-030-75245-3_18).

A General Description and Analysis of Matrices

Forgery Matrix. We can in general represent the matrix at the point of the forgery as

$$(c \ M_1 \ \dots \ M_m \ \hat{M}_1 \ \dots \ \hat{M}_m),$$

with $c = (c_1 \ \dots \ c_m \ c^* + \alpha_{pk})^T$, and

$$M_i = \begin{pmatrix} 0 & \dots & 0 \\ \vdots & \vdots & \vdots \\ 0 & \dots & 0 \\ \text{OneHot}(S_i) & & \\ 0 & \dots & 0 \\ \vdots & \vdots & \vdots \\ \beta_i^{(1)} & \dots & \beta_i^{(|S_i|)} \end{pmatrix}, \quad \hat{M}_i = \begin{pmatrix} 0 & \dots & 0 \\ \vdots & \vdots & \vdots \\ 0 & \dots & 0 \\ b_i \cdot \text{OneHot}(S_i) & & \\ 0 & \dots & 0 \\ \vdots & \vdots & \vdots \\ \gamma_i^{(1)} & \dots & \gamma_i^{(|S_i|)} \end{pmatrix}.$$

In the above, $\text{OneHot}(S_i)$ denotes the standard one-hot encoding of S_i as a subset of $[n]$, and the first non-null row of both M_i and $\hat{M}_i$ is the i -th one. We mark it as one-hot encoding, since honest, participating signers are represented by 1 and dishonest, or non-participating signers are marked as 0.

Case 1. The matrix obtained at the end of the first case of trivial non-linearity (Case 1) has the form

$$(L_1 \ \dots \ L_m \ \hat{L}_i),$$

with

$$L_i = (\text{OneHot}(i) \ \beta_i)^T, \quad \hat{L}_i = (b_i \cdot \text{OneHot}(i) \ \gamma_i)^T,$$

where $\text{OneHot}(i)$ denotes the binary vector of length $m-1$ where only the i -th entry is 1, and everything else is set to 0. For the last row to be a linear combination of the previous ones, the structure of the L_i columns imposes that $\gamma_i = b_i \beta_i$ for all i ’s. Therefore, since for the remaining γ_i , it holds, that $\gamma_i \beta_i \cdot b_i$, the full-rank is guaranteed.

Case 2. The general structure of the matrix obtained at the end of Case Item 2 is

$$(M_1 \dots M_m),$$

with M_i defined as above. We immediately note that the last row is a linear combination of the previous m ones iff there exists a constant $\kappa_i \in \mathbb{Z}_p$ such that $(\beta_1^{(i)} \dots \beta_{|S_i|}^{(i)}) = (\kappa_i \dots \kappa_i)$. Therefore, as soon as there exist two $j \neq k$ such that $\beta_k^{(i)} \neq \beta_j^{(i)}$, the above condition is contradicted, and the resulting matrix is full rank.

Final Matrix. After excluding Cases 1 and 2, we can verify that the general matrix has structure

$$(c \ N_1 \dots N_m \ \hat{N}_1 \dots \hat{N}_m),$$

with c defined as before and

$$N_i = \begin{pmatrix} 0 & \dots & 0 \\ \vdots & \vdots & \vdots \\ 0 & \dots & 0 \\ \text{OneHot}(S_i) & & \\ 0 & \dots & 0 \\ \vdots & \vdots & \vdots \\ \beta_i & \dots & \beta_i \end{pmatrix}, \quad \hat{N}_i = \begin{pmatrix} 0 & \dots & 0 \\ \vdots & \vdots & \vdots \\ 0 & \dots & 0 \\ b_i \cdot \text{OneHot}(S_i) & & \\ 0 & \dots & 0 \\ \vdots & \vdots & \vdots \\ b_i \beta_i & \dots & b_i \beta_i \end{pmatrix}.$$

From the latter, following the procedure outline in the main body, we obtain a *final matrix* having the following form

$$\begin{pmatrix} c_1 & 1 & 0 & \dots & 0 \\ c_2 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & & \ddots & \vdots \\ c_m & 0 & 0 & \dots & 1 \\ c^* + \alpha_{pk} & \beta_1 & \beta_2 & \dots & \beta_m \end{pmatrix}.$$